How to best predict insurance lapse - A Machine Learning Prediction Model Problem

STSCI 4990 Independent Project
Henry Tan

Agenda

1. Introduction and Motivation
2. Data Source
3. Exploratory Data Analysis
4. Data Cleaning
5. Feature Engineering and Model Fitting
6. Conclusion and Limitation

Motivation & Data Source

So, why do we even want to learn what factors are important in predicting insurance lapse?

Motivation & Data Source

Motivation 1: Strategic Business Necessity

Customer Lifetime Value (CLV) Optimization: *Identifying "at-risk" policyholders allows for proactive retention strategies. Because the cost of acquiring a new customer significantly exceeds the cost of retaining an existing one, precision in lapse prediction directly impacts profitability.*

Asset-Liability Management: *Lapses disrupt projected cash flows. Accurate models ensure that reserves and investment strategies are aligned with actual policyholder behavior, reducing liquidity risk.*

Risk-Based Pricing: *By understanding the drivers of lapse—such as premium sensitivity or policy duration—insurers can better refine their product offerings and pricing structures to meet market demand while maintaining stability.*

Motivation 2: Algorithm Exploration

Understanding Algorithmic Mechanics: *Explores the fundamental differences in how diverse algorithms approach the same problem → determine which mathematical approach best captures the nuances of insurance policyholder behavior.*

Strategic Feature Engineering & Optimization: *Researches and studies different mechanisms in handing messy dataset - including NA imputations, outliers handling and engineering features to match model assumptions.*

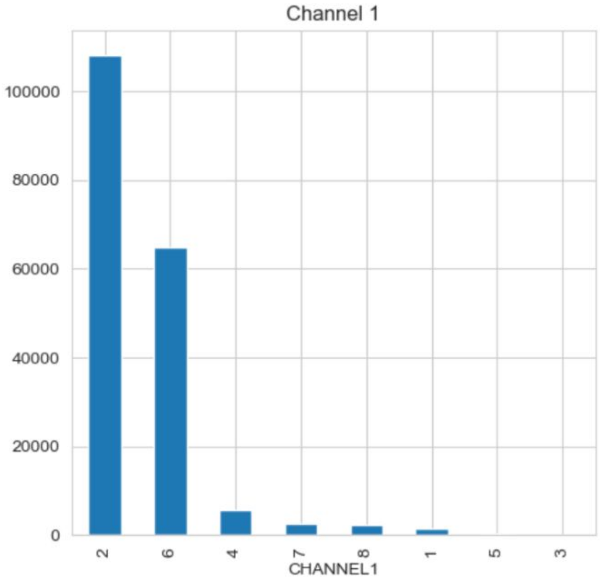
Decoding Model Interpretability: *Uses the outputs of different models to understand which factors are important in predicting insurance lapse, through feature importance or coefficients of the results.*

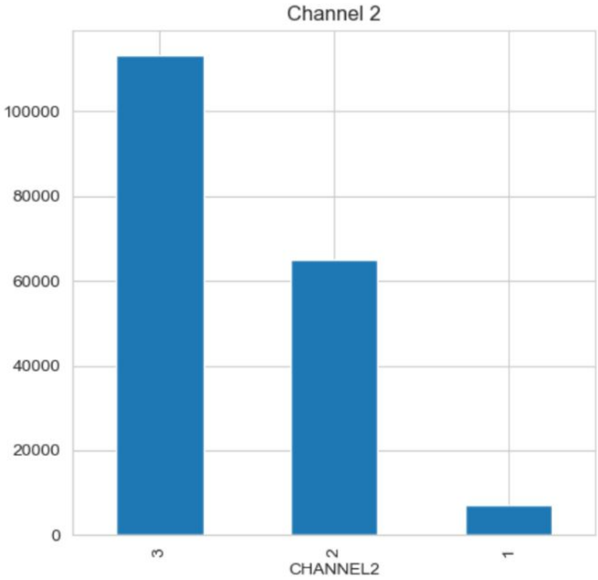
Data Source: Life Insurance Policy Data *(Kaggle - 185560 records)*

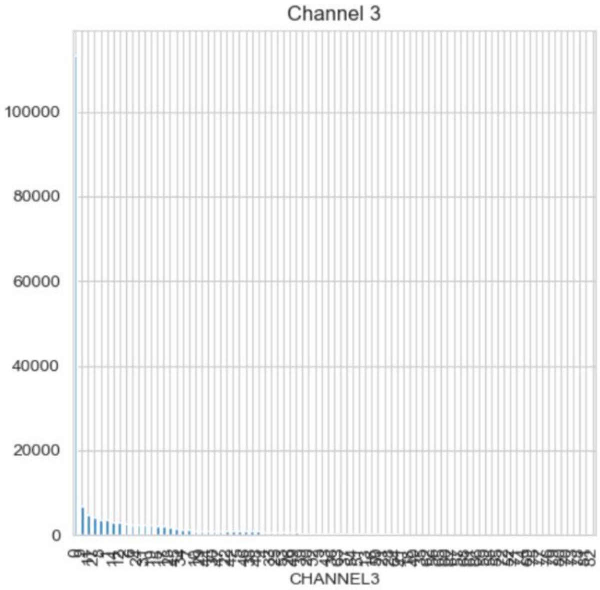
Exploratory Data Analysis

- *To explore the distribution of different variables*
- *To investigate the relationship between insurance lapse and other predictor variables*

Channels





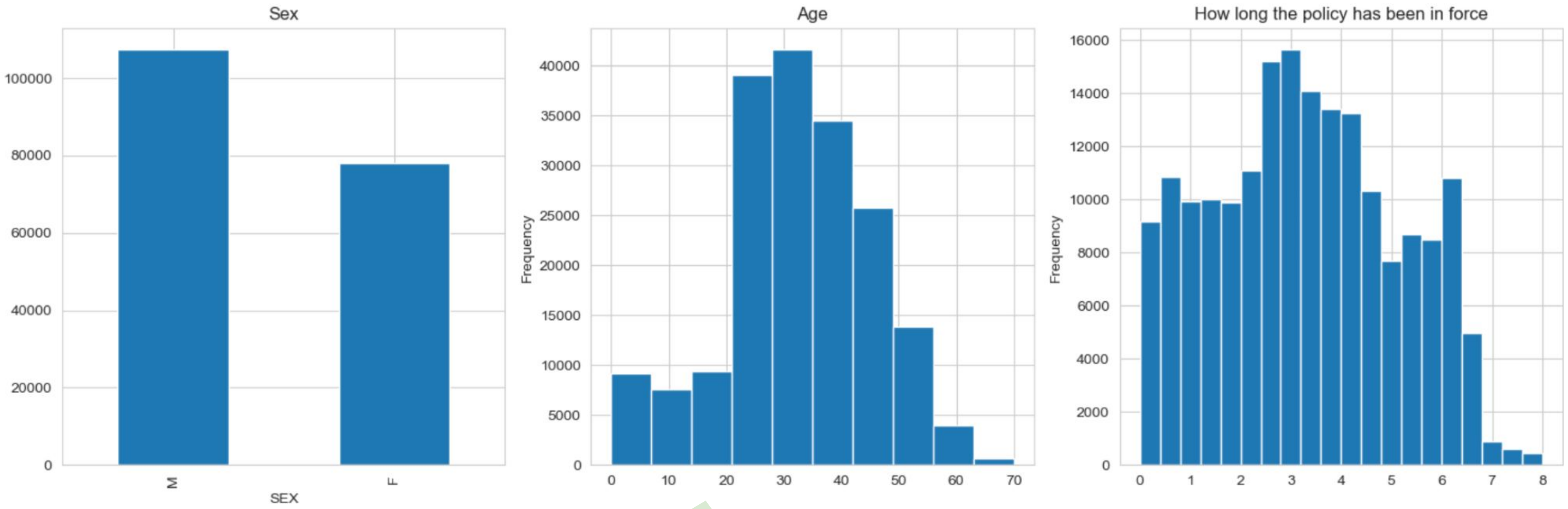


According to insurance context, insurance companies track Distribution Channels to see **which sales methods are most profitable or prone to "churn" (lapsing).** Having three separate channel columns suggests a hierarchical distribution model:

- (1) Channel 1 (The Broad Category): Likely represents the highest level of sales. For example: 1 = Agency, 2 = Direct-to-Consumer, 3 = Bancassurance (sold through banks).
- (2) Channel 2 (The Specific Entity): This might identify the specific branch or region. For instance, if Channel 1 is "Agency," Channel 2 might differentiate between "Independent Agents" vs. "Captive Agents."
- (3) Channel 3 (The Producer): This often identifies the specific broker group or the specific marketing campaign that brought the lead in.

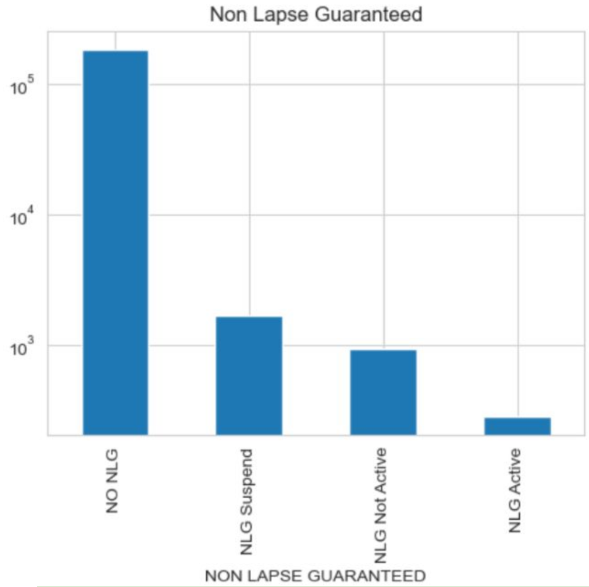
While this is the possible explanation and interpretation of each channel, the dataset lack clear definition in terms of the information these columns provided.
This might be useful in producing an interpretable model, but we should not leave them out since they might carry information that contributes to prediction accuracy.

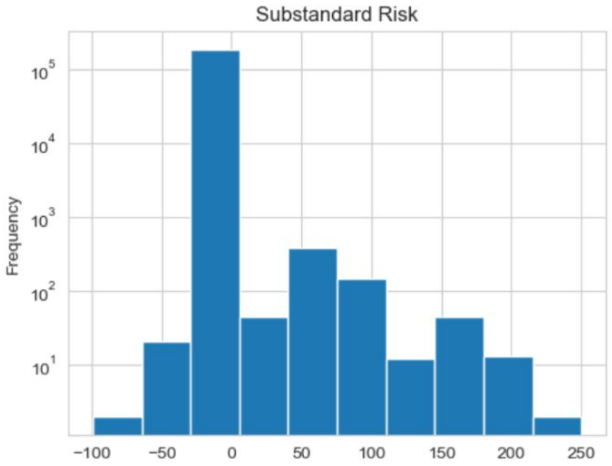
Sex, Age and Policy Year (Decimal)



Most of the policyholders are adults (20-60 years old)
Higher awareness about insurance protection

Policy Status, NLG, Substandard Risk





<u>Goals:</u> predict LAPSE vs NON-LAPSE (binary) → regrouping the variables into only two categories.

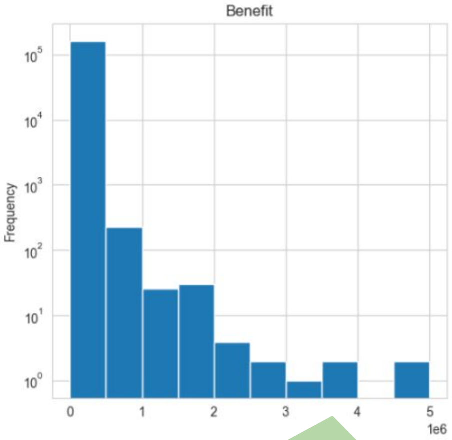
LAPSE: Lapse and Surrender, active and voluntary termination;
NON-LAPSE:INFORCE, EXPIRED, DEATH, passive and involuntary termination.

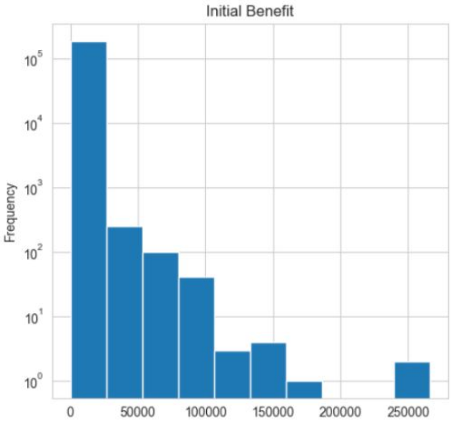
NLG: Non lapse guaranteed
A policy feature that ensures the death benefit remains in force even if the ***policy's cash value drops to zero,*** *provided a specific premium requirement is met.*

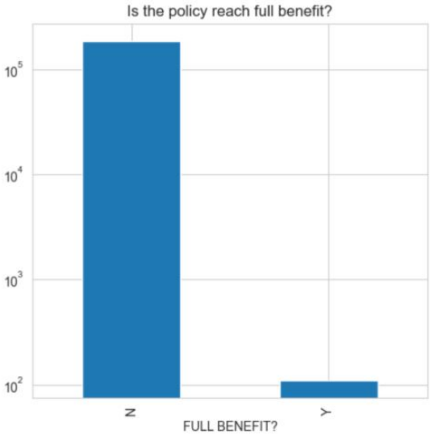
Substandard Risk: The extra mortality risk a policyholder carries compared to a "Standard" individual.

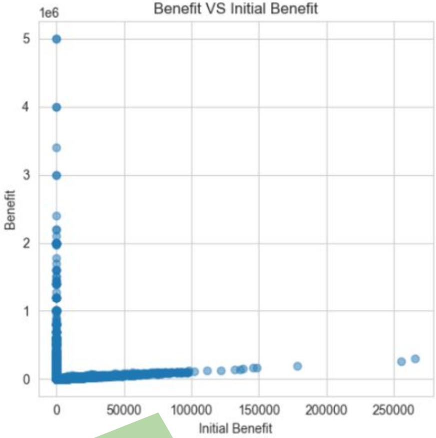
1.25% of the policies have negative standard risk score, which is a small proportion. Since substandard risk must be >=0, we **suspect negative values to be a typo, hence will drop these values.**

Benefit









***Very skewed** benefit and initial benefit - we need log transformation when doing analysis with logistics regression and KNN*

Very weird pattern
Most of the benefit cluttered at initial benefit =0
Most of the initial benefit cluttered at benefit = 0
Policies with high current benefits tend to have negligible initial benefits, and vice versa.

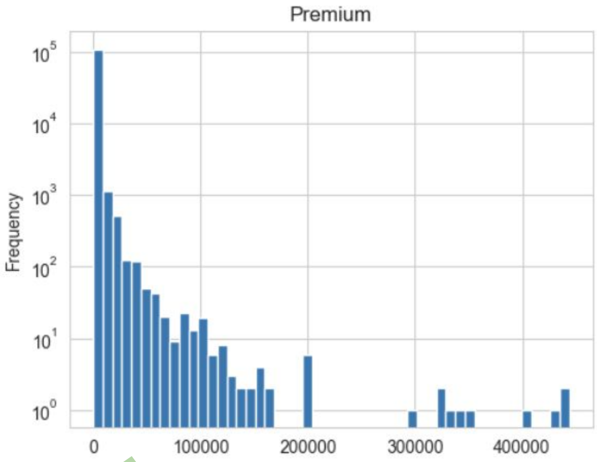
Initial Benefit

This is the original face amount of the insurance policy at the time it was first issued. It represents the starting death benefit or coverage level agreed upon when the contract was signed.

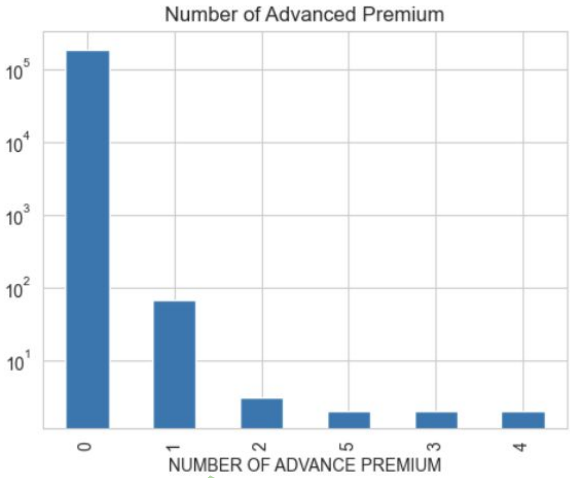
Benefit

This represents the **current death benefit or total coverage amount** as of the date of the data snapshot.
*In many life insurance products (like Universal Life or Whole Life), the benefit can **grow over time** due to paid-up additions, inflation adjustments, or accumulated dividends.*

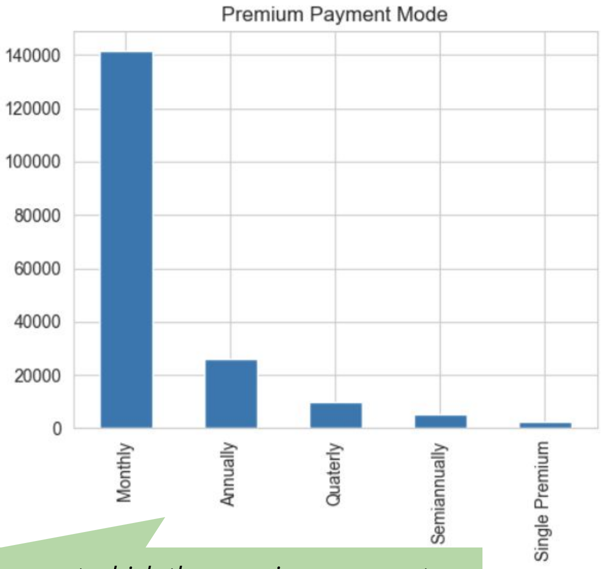
Premium



__Very skewed__ premium - we need log transformation when doing analysis with logistics regression and KNN

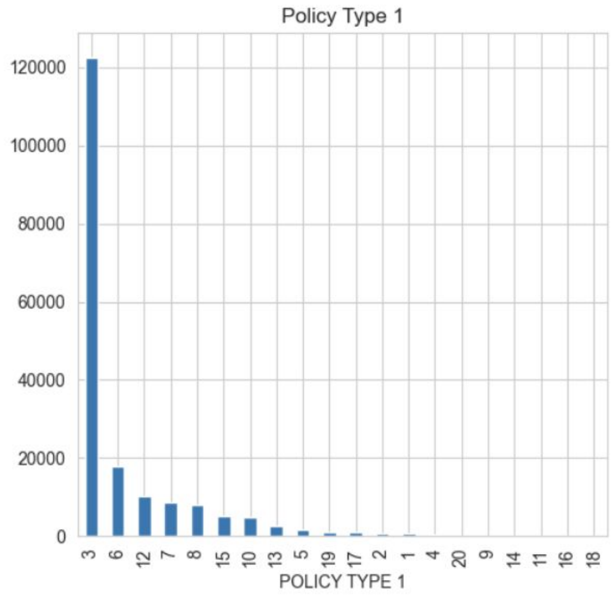


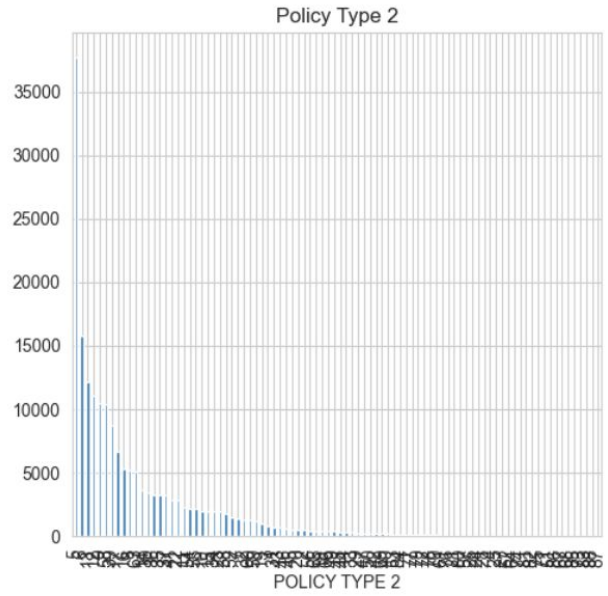
- How many **future premium payments** the policyholder has paid in advance of their actual due dates.
- Some policyholders **prefer to pay for several months or years** at once to ensure the policy doesn't lapse if they forget a payment.

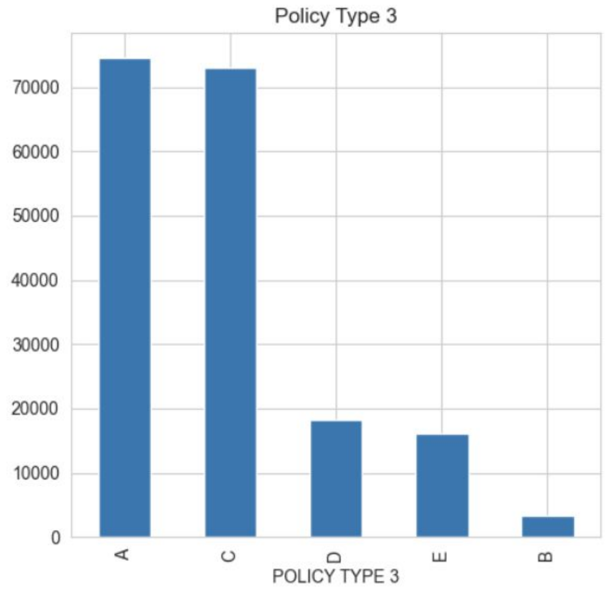


__Frequency__ at which the premium payments are made. Typically, policies paid "Annually" have lower lapse rates than those paid "Monthly," as there are fewer opportunities for a payment to be missed.

Policy Type







Insurance products are rarely just one thing; they are usually a base plan with various riders or **specific internal classifications.**

(1) Policy Type 1 (Product Class): This is likely the "Big Picture" product line. → Example: 1 = Term Life, 2 = Whole Life, 3 = Universal Life.

(2) Policy Type 2 (Sub-Product/Plan): Within a class, there are different variations. → Example: If Type 1 is "Term Life," Type 2 might be 1 = 10-year term or 2 = 20-year term.

(3) Policy Type 3 (Risk or Premium Class): This often correlates with the "Non-Lapse Guaranteed" (NLG) column or the "Benefit" amount. It might represent the specific underwriting tier or the specific dividend structure of the policy.

While this is the possible explanation and interpretation of each channel, the dataset lack clear definition in terms of the information these columns provided. This might be useful in producing an interpretable model, but we should not leave them out since they might carry information that contributes to prediction accuracy.

Data Cleaning

Cleaning and grouping variables to ensure good data structure before model fitting.

NA Checking and Imputation

CHANNEL1	0
CHANNEL2	0
CHANNEL3	0
ENTRY AGE	0
SEX	0
POLICY TYPE 1	0
POLICY TYPE 2	0
POLICY TYPE 3	0
PAYMENT MODE	0
POLICY STATUS	0
BENEFIT	23664
NON LAPSE GUARANTEED	0
SUBSTANDARD RISK	0
NUMBER OF ADVANCE PREMIUM	0
INITIAL BENEFIT	0
FULL BENEFIT?	0
POLICY YEAR (DECIMAL)	0
POLICY YEAR	0
PREMIUM	77127
ISSUE DATE	0

dtype: int64

NA Imputation Method

Create a dataset without NA values - split them into 70% of training and 30% of validation set

Within the validation set, we randomly select a few observations (10% from the validation set), acting them as the NAs. We trained imputation model with training data and use them to estimate the artificial NAs

Use MSE to measure accuracy

Imputation Methods Use:

1. Median Imputation
2. KNN Imputation (Cross validation to identify best K)
3. Quantile regression

```
--- MSE Comparison: PREMIUM ---
Median Baseline MSE:   13,630,353.44
KNN Imputation MSE: 9,340,850.82
Quantile Reg MSE:   8,524,639.07
```

```
--- MSE Comparison: BENEFIT ---
Median Baseline MSE:   6,754,695,669.97
KNN Imputation MSE: 5,657,737,305.84
Quantile Reg MSE:   4,808,801,494.70
```

Quantile Regression has the lowest MSE for Premium and Benefit NA imputation → will use Quantile Regression for overall NA imputation

NA Checking and Imputation

```python
# Based on the result above, we will use quantile regression to impute NAs for both Premium and Benefit
# We only use the columns involved in the calculation
cols_to_fix = ['PREMIUM', 'BENEFIT']
data_for_imputation = df[cols_to_fix].copy()

# Fit quantile regression models on the complete cases (non-NA) to predict missing values
mod_p = smf.quantreg('PREMIUM ~ BENEFIT', df).fit(q=0.5, max_iter=5000)
mod_b = smf.quantreg('BENEFIT ~ PREMIUM', df).fit(q=0.5, max_iter=5000)

# Fill in NAs values
df['PREMIUM'] = df['PREMIUM'].fillna(mod_p.predict(df))
df['BENEFIT'] = df['BENEFIT'].fillna(mod_b.predict(df))

# Final verification
print(f"Missing values in Premium: {df['PREMIUM'].isna().sum()}")
print(f"Missing values in Benefit: {df['BENEFIT'].isna().sum()}")
```

```
Missing values in Premium: 4315
Missing values in Benefit: 4315
```

After fitting the quantile regression, there are still 4315 missing values. This happens when the observations have both premium and benefit in NAs. Quantile regression need to rely on premium to estimate benefit and vice versa → hence do not work well for these rows.

Fit the 4315 variables with global median, for both Benefit and Premium.

Combining response variables

Steps

We only care about whether an individual will lapse or not *(binary problem) → combining the different categories in the LAPSE variables.*

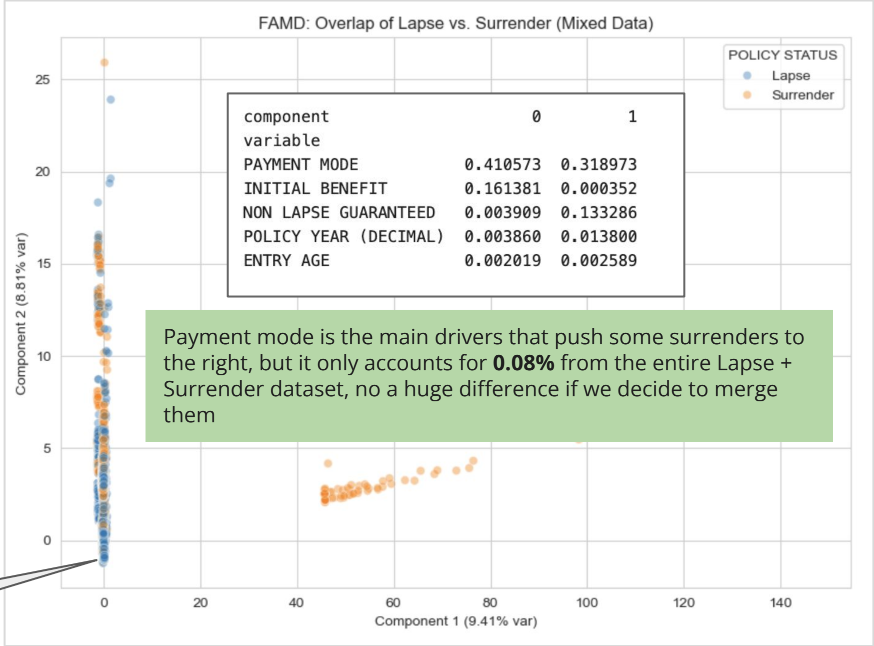
LAPSE: Lapse + Surrender
We need to conduct tests to make sure the individual profiles for lapse and surrender is very similar so they can be effectively being categorized as the same group.

<u>Method 1: Adversarial Classifier Test</u>
Train a model specifically to distinguish between Lapse and Surrender. If the model fails (close to 0.5), Lapse and Surrender are effectively the same to your classifier.

Result: AUC = 0.681 (not too far away from 0.5)

Method 2: Factor Analysis of Mixed Data

Reduces high dimensional data to 2 component to see the clusters of the idata



component	0	1
variable		
PAYMENT MODE	0.410573	0.318973
INITIAL BENEFIT	0.161381	0.000352
NON LAPSE GUARANTEED	0.003909	0.133286
POLICY YEAR (DECIMAL)	0.003860	0.013800
ENTRY AGE	0.002019	0.002589

Payment mode is the main drivers that push some surrenders to the right, but it only accounts for **0.08%** from the entire Lapse + Surrender dataset, no a huge difference if we decide to merge them

Feature space of lapse and surrender **highly overlap** and have similar profiles - merging is valid

Feature Engineering

Inspecting the data to ensure this fulfills basic assumption for model fitting

Data Inspection: Variables Available for Analysis

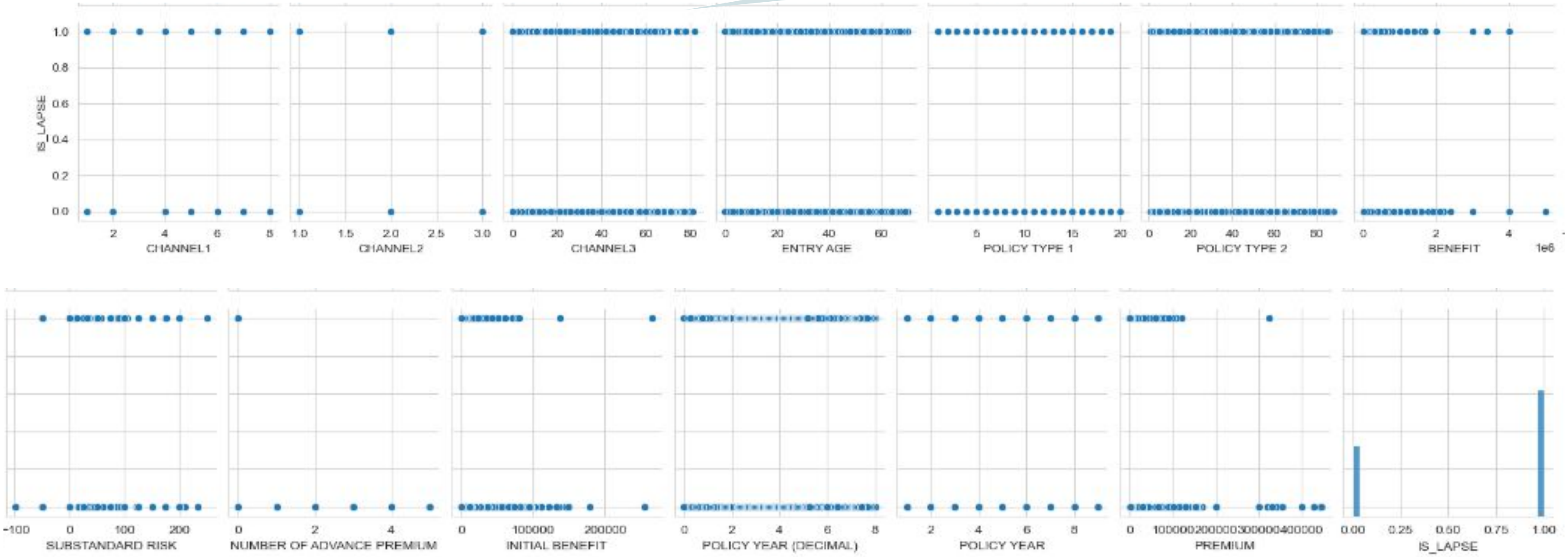
```
<class 'pandas.DataFrame'>
RangeIndex: 185560 entries, 0 to 185559
Data columns (total 21 columns):
 #   Column                     Non-Null Count   Dtype
---  ------                     --------------   -----
 0   CHANNEL1                   185560 non-null  int64
 1   CHANNEL2                   185560 non-null  int64
 2   CHANNEL3                   185560 non-null  int64
 3   ENTRY AGE                  185560 non-null  int64
 4   SEX                        185560 non-null  str
 5   POLICY TYPE 1              185560 non-null  int64
 6   POLICY TYPE 2              185560 non-null  int64
 7   POLICY TYPE 3              185560 non-null  str
 8   PAYMENT MODE               185560 non-null  str
 9   POLICY STATUS              185560 non-null  str
 10  BENEFIT                    185560 non-null  float64
 11  NON LAPSE GUARANTEED       185560 non-null  str
 12  SUBSTANDARD RISK           185560 non-null  float64
 13  NUMBER OF ADVANCE PREMIUM  185560 non-null  int64
 14  INITIAL BENEFIT            185560 non-null  float64
 15  FULL BENEFIT?              185560 non-null  str
 16  POLICY YEAR (DECIMAL)      185560 non-null  float64
 17  POLICY YEAR                185560 non-null  int64
 18  PREMIUM                    185560 non-null  float64
 19  ISSUE DATE                 185560 non-null  str
 20  IS_LAPSE                   185560 non-null  int64
dtypes: float64(5), int64(9), str(7)
memory usage: 29.7 MB
```

Response: IS_LAPSE
(1 = Lapse, 0 = Continue)

19 Predictors, 185560 Entries

Data Inspection: Correlation Plot

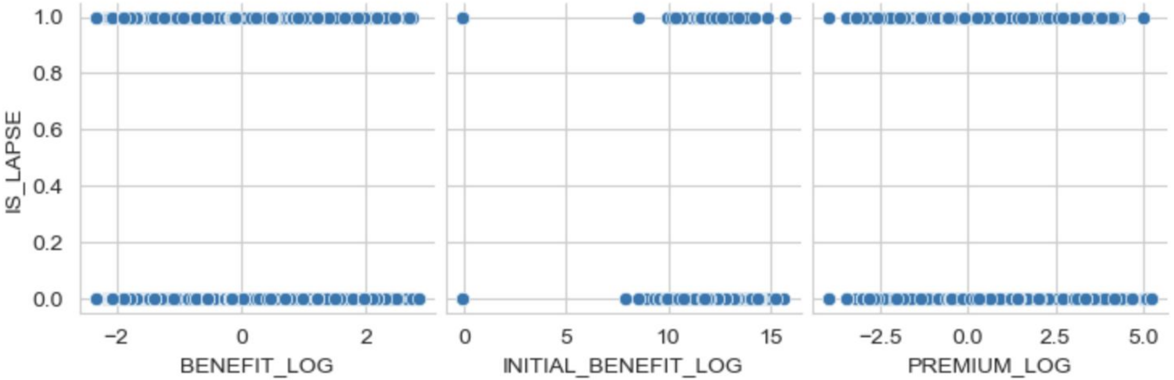
Due to the binary nature of the response variable, there is not a lot of significant trend appearing through the correlation plot. A lot of data are clustered together.



In particular, we know that we are trying to fit into a logistic regression model, and to ensure good performance, the relationship between the response and the predictors should be in sigmoid curve. However the relationship between IS_LAPSE and **Initial Benefit, Benefit and Premium appear to be inverted sigmoid.**

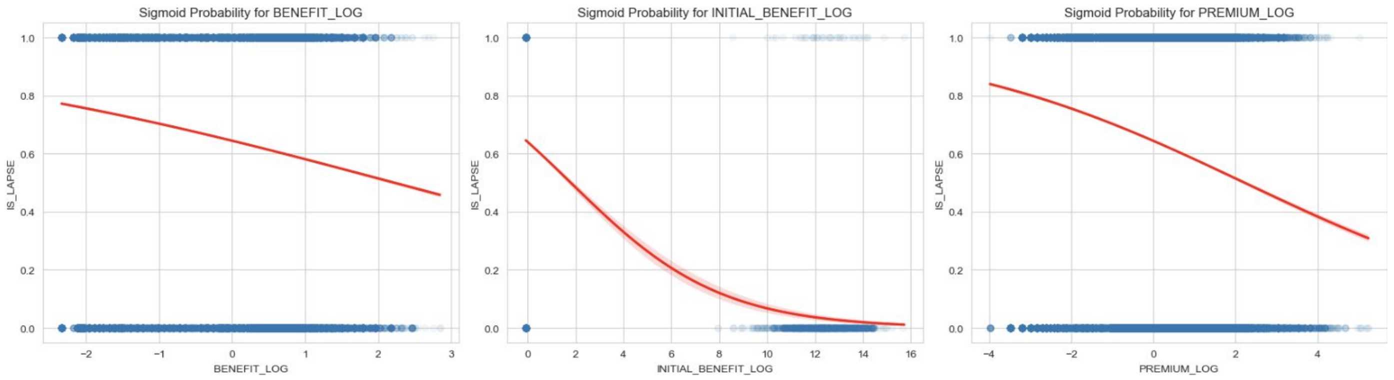
Feature Engineering: Correcting the Inverted Sigmoid shape

Log the Benefit, Initial Benefit and Premium because they are very right-skewed.
(Through our EDA)



Fit a sigmoid logistic curve
Better visualize and showcase that there is a inverted sigmoid relationship exists between.

This affects the performance of logistics regression because it tries to **fit a S shape onto the data,** and the inverted S shape makes the logistics predicts either a lot of them as 0 or 1, but **not all of them accurately.**

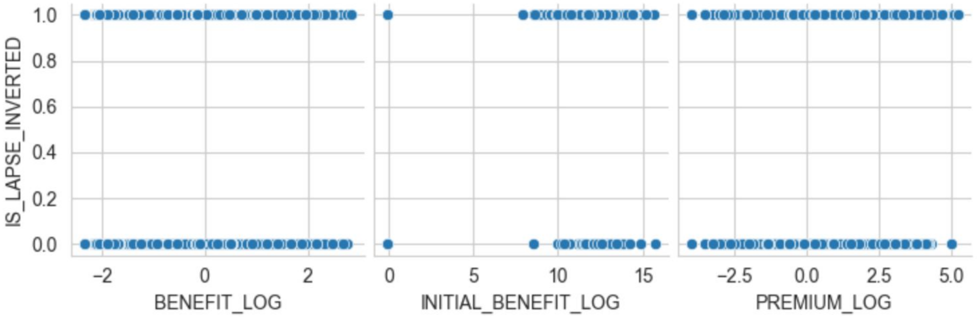


Feature Engineering: Invert Response Variable to form Sigmoid Shape

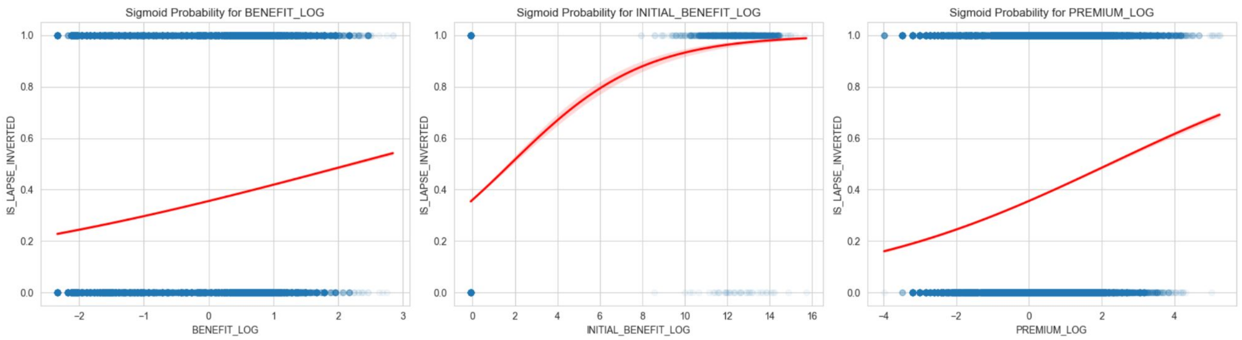
```
# The sigmoid curves confirm the inverted S-shape relationship between the log-transformed features and the probability of lapse.
# However, the sigmoid is still inverted.
# Let's try inverting my response variable
df['IS_LAPSE_INVERTED'] = 1 - df['IS_LAPSE']
```

```
# Transform the response variable
df['CONTINUE'] = 1 - df['IS_LAPSE']
df.drop(columns=['IS_LAPSE'], inplace=True)
```

Right now: 1 = CONTINUE, 0 = LAPSE
CONTINUE become our new response variable



Inverting response variable makes the S shape to be correct – easy for the prediction mechanism under logistic regression



Data Processing

Steps

Dropping irrelevant columns: Policy Status, Policy Year and Issue Date

1. *CONTINUE already captures the information in Policy Status*
2. *Policy Year has the same information as Policy Year (Decimal), we keep the decimal version for a more accurate information*
3. *Issue Date has no relevant information, and its information is captured by how long the policy has lasted – Policy Year (Decimal)*

Dropping rows with Substandard Risk <0, it is impossible for risk to be negative

Categorizing variables into continuous and categorical for Box Tidwell test *(checking linearity assumption between predictors and log-odds in logistic regression)*

Creating dummies for categorical variables

25 Predictors (After data processing + add dummies)

```
Data columns (total 25 columns):
 #   Column                                Non-Null Count   Dtype  
---  ------                                --------------   -----  
 0   CHANNEL1                              185537 non-null  float64
 1   CHANNEL2                              185537 non-null  float64
 2   CHANNEL3                              185537 non-null  float64
 3   ENTRY AGE                             185537 non-null  float64
 4   POLICY TYPE 1                         185537 non-null  float64
 5   POLICY TYPE 2                         185537 non-null  float64
 6   SUBSTANDARD RISK                      185537 non-null  float64
 7   PREMIUM                               185537 non-null  float64
 8   BENEFIT                               185537 non-null  float64
 9   POLICY YEAR (DECIMAL)                 185537 non-null  float64
 10  NUMBER OF ADVANCE PREMIUM             185537 non-null  float64
 11  INITIAL BENEFIT                       185537 non-null  float64
 12  SEX_M                                 185537 non-null  float64
 13  PAYMENT MODE_Monthly                  185537 non-null  float64
 14  PAYMENT MODE_Quaterly                 185537 non-null  float64
 15  PAYMENT MODE_Semiannually             185537 non-null  float64
 16  PAYMENT MODE_Single Premium           185537 non-null  float64
 17  POLICY TYPE 3_B                       185537 non-null  float64
 18  POLICY TYPE 3_C                       185537 non-null  float64
 19  POLICY TYPE 3_D                       185537 non-null  float64
 20  POLICY TYPE 3_E                       185537 non-null  float64
 21  NON LAPSE GUARANTEED_NLG Not Active   185537 non-null  float64
 22  NON LAPSE GUARANTEED_NLG Suspend      185537 non-null  float64
 23  NON LAPSE GUARANTEED_NO NLG           185537 non-null  float64
 24  FULL BENEFIT?_Y                       185537 non-null  float64
dtypes: float64(25)
```

Data Splitting for Machine Learning

Initial Dataset
185560 Observations

Training Data
85%

Testing Data
15%

Training Set
70%

Validation Set
30%

Split the data into training and testing set according to **85-15 ratio.** The training set is used to train the model, whereas the testing set acts as unseen and unknown data, useful for our to test the out of sample performance of our model to ensure a more robust analysis.

Then, we will further split the training set into training and validation set, according to a **70-30 ratio**. The training set is used to train the model, whereas the validation set is used to test the current model performance, assuming that the training set is the data we collected.

```
Data Split Complete:
Train: 110394
Val: 47312
Test: 27831
```

Checking Multicollinearity

--- Variance Inflation Factor (VIF) ---

	Feature	VIF
0	CHANNEL1	3.718692
1	CHANNEL2	2.679632
2	CHANNEL3	2.093053
3	ENTRY AGE	1.040134
4	POLICY TYPE 1	12.354155
5	POLICY TYPE 2	11.381739
6	SUBSTANDARD RISK	1.013375
7	PREMIUM	1.297758
8	BENEFIT	1.338680
9	POLICY YEAR (DECIMAL)	1.682636
10	NUMBER OF ADVANCE PREMIUM	1.003361
11	INITIAL BENEFIT	1.427098
12	SEX_M	1.019880
13	PAYMENT MODE_Monthly	1.692614
14	PAYMENT MODE_Quaterly	1.369073
15	PAYMENT MODE_Semiannually	1.188937
16	PAYMENT MODE_Single Premium	2.095477
17	POLICY TYPE 3_B	1.377880
18	POLICY TYPE 3_C	1.711746
19	POLICY TYPE 3_D	1.223879
20	POLICY TYPE 3_E	1.562264
21	NON LAPSE GUARANTEED_NLG Not Active	4.411967
22	NON LAPSE GUARANTEED_NLG Suspend	7.214744
23	NON LAPSE GUARANTEED_NO NLG	10.582430
24	FULL BENEFIT?_Y	1.009523

We performed VIF onto the *training set* to avoid data leakage.

This method aims to discover any closely related variables, which might inflate the variance of our coefficients and make them unstable.

Ideally, we hope our features are **very uncorrelated between each other so each of them could provide unique and distinct information to predicting responses.**

Based on the results above, we saw a large VIF for **POLICY TYPE 1 and 2**, indicating that there is a strong correlation between these two variables, and they likely contribute the same information to predicting the response. Since that's the case, we will drop POLICY TYPE 2.

In addition, there is a high correlation between 2 levels, **SUSPEND and NO NLG in the NON LAPSE GUARANTEED** categories, indicating that both levels might be conveying the same information. Since they are both technically 'Inactive', <u>*we could group them under one categories - Inactive.*</u>

Eliminating Multicollinearity

All VIF is lower than 5, not a big concern, no huge multicollinearity in the data now → **22 variables**

--- Variance Inflation Factor (VIF) ---

	Feature	VIF
0	CHANNEL1	3.718692
1	CHANNEL2	2.679632
2	CHANNEL3	2.093053
3	ENTRY AGE	1.040134
4	POLICY TYPE 1	12.354155
5	POLICY TYPE 2	11.381739
6	SUBSTANDARD RISK	1.013375
7	PREMIUM	1.297758
8	BENEFIT	1.338680
9	POLICY YEAR (DECIMAL)	1.682636
10	NUMBER OF ADVANCE PREMIUM	1.003361
11	INITIAL BENEFIT	1.427098
12	SEX_M	1.019880
13	PAYMENT MODE_Monthly	1.692614
14	PAYMENT MODE_Quaterly	1.369073
15	PAYMENT MODE_Semiannually	1.188937
16	PAYMENT MODE_Single Premium	2.095477
17	POLICY TYPE 3_B	1.377880
18	POLICY TYPE 3_C	1.711746
19	POLICY TYPE 3_D	1.223879
20	POLICY TYPE 3_E	1.562264
21	NON LAPSE GUARANTEED_NLG Not Active	4.411967
22	NON LAPSE GUARANTEED_NLG Suspend	7.214744
23	NON LAPSE GUARANTEED_NO NLG	10.582430
24	FULL BENEFIT?_Y	1.009523

Transformation

Combining Inactive column that combine NLG Not Active, No NLG and Suspend

Remove Policy Type 2 variable

--- Variance Inflation Factor (VIF) ---

	Feature	VIF
0	CHANNEL1	3.698844
1	CHANNEL2	2.676044
2	CHANNEL3	2.092345
3	ENTRY AGE	1.039110
4	POLICY TYPE 1	1.727605
5	SUBSTANDARD RISK	1.013334
6	PREMIUM	1.297349
7	BENEFIT	1.338178
8	POLICY YEAR (DECIMAL)	1.666447
9	NUMBER OF ADVANCE PREMIUM	1.003178
10	INITIAL BENEFIT	1.426966
11	SEX_M	1.019796
12	PAYMENT MODE_Monthly	1.674164
13	PAYMENT MODE_Quaterly	1.362629
14	PAYMENT MODE_Semiannually	1.187923
15	PAYMENT MODE_Single Premium	2.086174
16	POLICY TYPE 3_B	1.376234
17	POLICY TYPE 3_C	1.491861
18	POLICY TYPE 3_D	1.222646
19	POLICY TYPE 3_E	1.306500
20	FULL BENEFIT?_Y	1.008220
21	NON LAPSE GUARANTEED_Inactive	1.002087

Model Fitting

Fitting with different machine learning models and identifying models with highest prediction accuracy

Logistics Regression - Assumption Check

Assumption Check

(1) Binary Outcome (Target only has 0 and 1)

(2) Independence of Errors *(Hard to check here)*

(3) Linearity (Predictors are linear to Log-Odds) - *Box Tidwell Test*

We only need to check the relationship between continuous features and the log-odds

H0: There is linear relationship between the continuous features and log-odds

Box-Tidwell tests the linearity by adding an **interaction log term (xlogx) i**nto the model.

If the original relationship is already linear, the model won't need this new interaction term to explain the data, and the p-value of the log term won't be significant.

No meaningful interpretation for the p-value of predictor themselves.

```
CONTINUE
0    0.641828
1    0.358172
Name: proportion, dtype: float64
```

```
const                               4.939036e-23
CHANNEL1                            5.220348e-32
CHANNEL2                            1.630302e-41
CHANNEL3                            5.357578e-01
ENTRY AGE                          1.067319e-198
POLICY TYPE 1                       2.848439e-11
SUBSTANDARD RISK                    7.495981e-01
PREMIUM                             2.556597e-34
BENEFIT                             6.467717e-02
POLICY YEAR (DECIMAL)               0.000000e+00
NUMBER OF ADVANCE PREMIUM           9.997491e-01
INITIAL BENEFIT                     7.071831e-13
CHANNEL1_log                        2.474464e-29
CHANNEL2_log                        2.627014e-40
CHANNEL3_log                        3.973045e-01
ENTRY AGE_log                      8.864672e-212
POLICY TYPE 1_log                   2.557913e-08
SUBSTANDARD RISK_log                8.557623e-01
PREMIUM_log                         1.372743e-32
BENEFIT_log                         2.416965e-01
POLICY YEAR (DECIMAL)_log           0.000000e+00
NUMBER OF ADVANCE PREMIUM_log       9.999136e-01
INITIAL BENEFIT_log                 8.081924e-12
dtype: float64
```

Based on the result above, the p-values of most of the log variables is very small (significant), providing sufficient evidence that **most of the features and log-odds are not linearly related.**

The insignificant/linearly related variables with response is CHANNEL3, SUBSTANDARD RISK, BENEFIT, NUMBER OF ADVANCE PREMIUM.

Even though this violates logistic regression, we can still conduct this test as our **baseline** to see the performance.

Logistics Regression *(Base Model)*

Steps

- Log transforms skewed variables: Benefit, Premium and Initial Benefit
- Apply standard scaler to scale and standardize the predictors since logistic regression is very sensitive to variables with very large figures
- Fit the logistic regression model

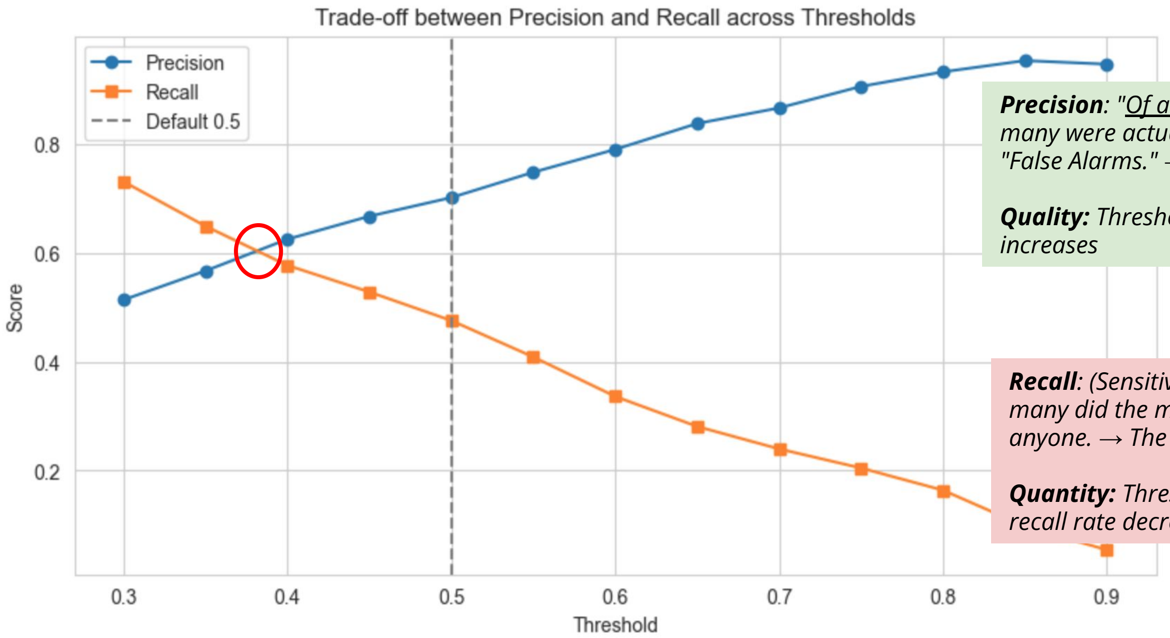
	coef	std err	z	P>|z|	[0.025	0.975]
const	0.0045	938.944	4.77e-06	1.000	-1840.292	1840.301
CHANNEL1	0.0269	0.014	1.930	0.054	-0.000	0.054
CHANNEL2	-0.1226	0.012	-10.148	0.000	-0.146	-0.099
CHANNEL3	0.0200	0.010	2.074	0.038	0.001	0.039
ENTRY AGE	0.0547	0.007	7.471	0.000	0.040	0.069
POLICY TYPE 1	0.2348	0.009	25.842	0.000	0.217	0.253
SUBSTANDARD RISK	0.0266	0.007	3.768	0.000	0.013	0.040
POLICY YEAR (DECIMAL)	-0.6399	0.009	-67.913	0.000	-0.658	-0.621
NUMBER OF ADVANCE PREMIUM	0.8453	5.04e+04	1.68e-05	1.000	-9.88e+04	9.88e+04
SEX_M	-0.0548	0.014	-3.785	0.000	-0.083	-0.026
PAYMENT MODE_Monthly	-1.1235	0.021	-52.473	0.000	-1.165	-1.082
PAYMENT MODE_Quaterly	-0.9734	0.036	-27.134	0.000	-1.044	-0.903
PAYMENT MODE_Semiannually	-0.7758	0.045	-17.126	0.000	-0.865	-0.687
PAYMENT MODE_Single Premium	1.0866	0.206	5.273	0.000	0.683	1.490
POLICY TYPE 3_B	1.8682	0.070	26.579	0.000	1.730	2.006
POLICY TYPE 3_C	0.2431	0.021	11.539	0.000	0.202	0.284
POLICY TYPE 3_D	0.5247	0.027	19.755	0.000	0.473	0.577
POLICY TYPE 3_E	-0.4677	0.039	-12.065	0.000	-0.544	-0.392
FULL BENEFIT?_Y	0.6653	0.346	1.925	0.054	-0.012	1.343
NON LAPSE GUARANTEED_Inactive	0.1526	0.212	0.720	0.471	-0.263	0.568
BENEFIT_LOG	-0.0213	0.010	-2.036	0.042	-0.042	-0.001
PREMIUM_LOG	0.2503	0.013	19.552	0.000	0.225	0.275
INITIAL BENEFIT_LOG	0.1065	0.024	4.509	0.000	0.060	0.153

Most variables are significant, except

1. NLG Inactive
2. Number of advance premium

This can only be a baseline model and the validity is not 100% due to the violation of the linearity assumptions

Logistics Regression *(Base Model)* - Prediction Model Performance



***Precision**: "<u>Of all the instances the model predicted as positive</u>, how many were actually positive?"It is about being accurate and avoiding "False Alarms." → when the model says Yes, you can usually trust it*

***Quality:** Threshold increase, more strict in selecting CONTINUE, precision increases*

***Recall**: (Sensitivity): "<u>Of all the actual positive instances that exist,</u> how many did the model find?"It is about completeness and not missing anyone. → The model is good at finding target*

***Quantity:** Threshold increase, less chance to include all CONTINUE, recall rate decreases*

Exact intersection threshold: 0.3816
This means when the estimated probability is > 0.3816, we can categorize the model as CONTINUE, otherwise as LAPSE → balance point between getting less false alarm but at the same time not ignoring all of the other correct actual CONTINUE

Logistics Regression *(Base Model)* - Prediction Model Performance

True Negative	False Positive
False Negative	True Positive

False Positive is more expensive than False Negative

Balanced Threshold: 0.3816

```
--- Final Test Set Results (Threshold 0.3816) ---
Misclassification Error Rate: 0.2810
Log-Loss:                     0.5393
ROC AUC Score:                0.7599

Confusion Matrix:
[[13899  3964]
 [ 3857  6111]]

Classification Report:
              precision    recall  f1-score   support

           0       0.78      0.78      0.78     17863
           1       0.61      0.61      0.61      9968

    accuracy                           0.72     27831
   macro avg       0.69      0.70      0.70     27831
weighted avg       0.72      0.72      0.72     27831
```

Bayes Classifier: 0.5

```
--- Final Test Set Results (Threshold 0.5000) ---
Misclassification Error Rate: 0.2583
Log-Loss:                     0.5393
ROC AUC Score:                0.7599

Confusion Matrix:
[[15855  2008]
 [ 5180  4788]]

Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.89      0.82     17863
           1       0.70      0.48      0.57      9968

    accuracy                           0.74     27831
   macro avg       0.73      0.68      0.69     27831
weighted avg       0.74      0.74      0.73     27831
```

In the insurance context, **recall rate might be more important than precision rate.** Recall rate specifies how many individuals do we manage to catch, whereas precision is specifying out of all the prediction that model flagged, how many is correct. The former is obviously more expensive because catching as many individuals who is likely to lapse as possible help us to craft targeted marketing materials to nudge the individuals stay in the policies to have enough reserves in place. → **0.5 threshold**

Logistics Regression with Feature Selection - LASSO

We suspect we might not need all of the **22 features** in the original models, hence we conducted feature selection

Using Cross Validation, we selected the penalty term lambda in our L1 regularization LASSO.

C = 1/lambda

```
Best C found: 0.3593813663804626
Features kept: 22
```

When applying L1 penalty to LASSO, no features are being eliminated → all features are significant and contribute some information in predicting the final insurance lapse status.

Balanced Threshold: 0.3806

```
--- LASSO Final Test Results (Threshold 0.3806) ---
Features used: 22 out of 22
Misclassification Error Rate: 0.2814
ROC AUC Score:                0.7599

Confusion Matrix:
[[13873  3990]
 [ 3843  6125]]

Classification Report:
              precision    recall  f1-score   support

           0       0.78      0.78      0.78     17863
           1       0.61      0.61      0.61      9968

    accuracy                           0.72     27831
   macro avg       0.69      0.70      0.69     27831
weighted avg       0.72      0.72      0.72     27831
```

Bayes Classifier: 0.5158

```
--- LASSO Final Test Results (Threshold 0.5158) ---
Features used: 22 out of 22
Misclassification Error Rate: 0.2574
ROC AUC Score:                0.7599

Confusion Matrix:
[[16069  1794]
 [ 5369  4599]]

Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.90      0.82     17863
           1       0.72      0.46      0.56      9968

    accuracy                           0.74     27831
   macro avg       0.73      0.68      0.69     27831
weighted avg       0.74      0.74      0.73     27831
```

Trial and Error: 0.49 to 0.55 at 0.0001 interval

Logistics Regression with Backward Subset Selection

We suspect we might not need all of the **22 features** in the original models, hence we conducted feature selection

Again, no features are eliminated through backward subset selection

The result is the same as if we fit the original model with logistics regression

No improvement in feature selection

Balanced Threshold: 0.3806

```
====================================================
--- BACKWARD SELECTION (AIC) FINAL RESULTS ---
Final Features: 22
Optimal Equilibrium Threshold: 0.3806
Misclassification Error Rate:  0.2814
ROC AUC Score:                 0.7599
====================================================

Confusion Matrix:
[[13870  3993]
 [ 3840  6128]]

Classification Report:
              precision    recall  f1-score   support

           0       0.78      0.78      0.78     17863
           1       0.61      0.61      0.61      9968

    accuracy                           0.72     27831
   macro avg       0.69      0.70      0.69     27831
weighted avg       0.72      0.72      0.72     27831
```

Bayes Classifier: 0.5

```
====================================================
--- BACKWARD SELECTION (AIC) FINAL RESULTS ---
Final Features: 22
Optimal Equilibrium Threshold: 0.5
Misclassification Error Rate:  0.2583
ROC AUC Score:                 0.7599
====================================================

Confusion Matrix:
[[15855  2008]
 [ 5180  4788]]

Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.89      0.82     17863
           1       0.70      0.48      0.57      9968

    accuracy                           0.74     27831
   macro avg       0.73      0.68      0.69     27831
weighted avg       0.74      0.74      0.73     27831
```

Logistics Regression with Ramsey Reset Test

Algorithm

Tests whether **non-linear combinations** of your independent variables have any power in explaining the response variable.

1. Fit the original model to get the predicted values
2. Rerun the regression, but add powers to those predicted values as new independent variables
3. **Likelihood ratio test** to test significance → if yes, there is non-linear relationship involved

```
--- Ramsey RESET Test for Logistic Regression ---
Likelihood Ratio Statistic: 539.5159
Degrees of Freedom:         1
P-value:                    0.0000

Result: Reject H0. The model is likely MISSPECIFIED.
Consider adding interaction terms or polynomial features.
```

Solution:
Adding interaction terms for all the variables
253 variables

Fit the **253 variables** into logistics regression, assess AUC-ROC (maximize) and False Positive (minimize)

Fit the 253 variables into LASSO + logistics regression, assess AUC-ROC (maximize) and False Positive (minimize)

Logistics Regression with Interaction Terms

Fit the **253 variables** into logistics regression, assess AUC-ROC (maximize) and False Positive (minimize)

*Logistics regression with full model: **0.7599***

```
==========================================
LASSO INTERACTION MODEL PERFORMANCE
==========================================
Test ROC AUC Score: 0.7700
Test Accuracy:    0.7474
Misclassification Error Rate: 0.2526

Confusion Matrix:
[[16090  1773]
 [ 5257  4711]]

Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.90      0.82     17863
           1       0.73      0.47      0.57      9968

    accuracy                           0.75     27831
   macro avg       0.74      0.69      0.70     27831
weighted avg       0.74      0.75      0.73     27831
```

Adding interaction term form non-linear relationships that improve ROC-AUC, essential to improve prediction

Fit the 253 variables into LASSO + logistics regression, assess AUC-ROC (maximize) and False Positive (minimize)

```
==========================================
LASSO INTERACTION MODEL PERFORMANCE
==========================================
Test ROC AUC Score: 0.7529
Test Accuracy:    0.7360
Misclassification Error Rate: 0.2640

Confusion Matrix:
[[15897  1966]
 [ 5382  4586]]

Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.89      0.81     17863
           1       0.70      0.46      0.56      9968

    accuracy                           0.74     27831
   macro avg       0.72      0.68      0.68     27831
weighted avg       0.73      0.74      0.72     27831
```

Lasso selects **235 features out of 253** - only minor feature selection, and performing worse than the original model (slightly lower ROC-AUC score)

```
Training ROC AUC: 0.7498
Test ROC AUC:     0.7529
AUC Gap:          -0.0031

Result: The model generalizes well to new data.
```

Cross validation to make sure no overfitting

K-Nearest Neighbour Classification

Steps

Log transforms skewed variables: Benefit, Premium and Initial Benefit

Apply standard scaler to scale and standardize the predictors since KNN is very sensitive to variables with very large figures

Use GridSearchCV and ROC-AUC as the criteria to select the K that yields the best results (highest ROC-AUC)

Search through [5, 11, 21, 31, 41, 51, 61, 71, 81, 91, 101]
With uniform weighting

K = 81 (*fitting 5 folds for each of 22 candidates, 110 fits)*

To ensure more robust estimation, we estimate the probabilities of classifying into either class, and from there figuring out **a threshold** we can use to categorize the response.

Precision-Recall Balanced Threshold: **0.3704**

K-Nearest Neighbour Classification

Under KNN methods, we obtained the similar results as logistic regression.

The threshold we obtained is around 0.37 to 0.38.

They have the balanced precision and recall rate, but have a higher false positive rate compared to using Bayes classifier threshold 0.5.

Logistics regression with full model interaction: ***0.77 → KNN performed slightly better at 0.7756 ROC-AUC***

Balanced Threshold: 0.3704

```
--- KNN Final Test Results (Threshold 0.3704) ---
Misclassification Error Rate: 0.2710
ROC AUC Score:                0.7756

Confusion Matrix:
[[14105  3758]
 [ 3783  6185]]

Classification Report:
              precision    recall  f1-score   support

           0       0.79      0.79      0.79     17863
           1       0.62      0.62      0.62      9968

    accuracy                           0.73     27831
   macro avg       0.71      0.71      0.71     27831
weighted avg       0.73      0.73      0.73     27831
```

Bayes Classifier: 0.51

```
--- KNN Final Test Results (Threshold 0.51) ---
Misclassification Error Rate: 0.2518
ROC AUC Score:                0.7756

Confusion Matrix:
[[16359  1504]
 [ 5505  4463]]

Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.92      0.82     17863
           1       0.75      0.45      0.56      9968

    accuracy                           0.75     27831
   macro avg       0.75      0.68      0.69     27831
weighted avg       0.75      0.75      0.73     27831
```

Trial and Error: 0.49 to 0.55 at 0.01 interval

Random Forest Classifier

Non-parametric method, handles multicollinearity and less sensitive to outliers

500 trees, Balanced class weight

Each of those trees pays "extra attention" to the minority class (the Lapses) during the bagging process.

Bagging:

1. Creates multiple parallel decision trees
2. Train each one on a random subset of the data (bootstraping)
3. Each tree is trained independently
4. **Final prediction:** majority vote for classification of all trees

***KNN performed better at 0.7756 ROC-AUC** - Random Forest is not a suitable prediction model here*

Balanced Threshold: 0.4160

```
--- RFC Final Test Results (Threshold 0.4160) ---
Misclassification Error Rate: 0.2797
ROC AUC Score:                0.7693

Confusion Matrix:
[[13945  3918]
 [ 3866  6102]]

Classification Report:
              precision    recall  f1-score   support

           0       0.78      0.78      0.78     17863
           1       0.61      0.61      0.61      9968

    accuracy                           0.72     27831
   macro avg       0.70      0.70      0.70     27831
weighted avg       0.72      0.72      0.72     27831
```

Bayes Classifier: 0.50

```
--- RFC Final Test Results (Threshold 0.5) ---
Misclassification Error Rate: 0.2662
ROC AUC Score:                0.7693

Confusion Matrix:
[[15029  2834]
 [ 4575  5393]]

Classification Report:
              precision    recall  f1-score   support

           0       0.77      0.84      0.80     17863
           1       0.66      0.54      0.59      9968

    accuracy                           0.73     27831
   macro avg       0.71      0.69      0.70     27831
weighted avg       0.73      0.73      0.73     27831
```

Trial and Error: 0.49 to 0.55 at 0.01 interval

XGBoost (eXtreme Gradient Boosting)

Non-parametric method, handles multicollinearity and less sensitive to outliers
-boosting and pruning make it more efficient

Boosting and Pruning:

1. Build trees sequentially
2. Each new tree is trained to predict the residuals of the previous trees
3. Use gradient descent minimizing log loss to improve prediction
4. **Backward pruning approach**, growing the tree to its maximum depth first and then starts pruning nodes backward from the bottom up if the gain at a specific split does not exceed a certain threshold.

Significantly higher ROC-AUC (0.7963) compared to KNN (0.7756), while maintaining similar False Positive (1539 VS 1504)

Balanced Threshold: 0.3578

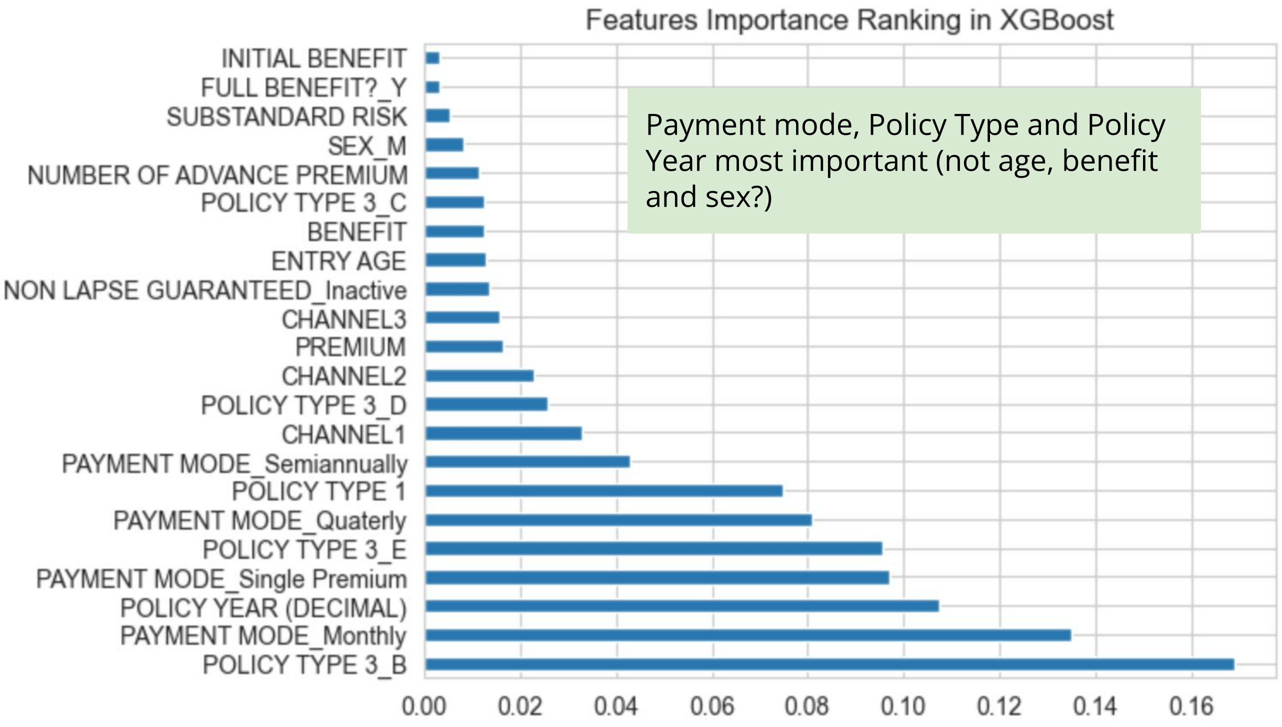
```
--- XGBoost Final Test Results (Threshold 0.3578) ---
Misclassification Error Rate: 0.2594
ROC AUC Score:                0.7963

Confusion Matrix:
[[14237  3626]
 [ 3594  6374]]

Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.80      0.80     17863
           1       0.64      0.64      0.64      9968

    accuracy                           0.74     27831
   macro avg       0.72      0.72      0.72     27831
weighted avg       0.74      0.74      0.74     27831
```

Bayes Classifier: 0.495

```
--- XGBoost Final Test Results (Threshold 0.495) ---
Misclassification Error Rate: 0.2399
ROC AUC Score:                0.7963

Confusion Matrix:
[[16324  1539]
 [ 5137  4831]]

Classification Report:
              precision    recall  f1-score   support

           0       0.76      0.91      0.83     17863
           1       0.76      0.48      0.59      9968

    accuracy                           0.76     27831
   macro avg       0.76      0.70      0.71     27831
weighted avg       0.76      0.76      0.74     27831
```

Trial and Error: 0.49 to 0.55 at 0.01 interval

XGBoost Feature Importance Ranking



Payment mode, Policy Type and Policy Year most important (not age, benefit and sex?)

Significant Predictors

Policy Type 3_B: This category has a very different lapse behavior compared t the others → *investigate if this policy type is a legacy product, a specific promotion, or a high-risk category that requires a tailored retention strategy.*

Payment Mode: *Monthly payers typically show higher lapse volatility (more "opportunities" to stop paying), while Single Premium payers are often "locked in."*

Policy Year: Customers likely to re-evaluate their coverage at 1-2y mark to see if continue the policy

Initial Benefit, Benefit and Premium are not really contributing to the prediction

How a customer pays and what product they bought is much more predictive of a lapse than how much the policy is worth.

Light GBM (Light Gradient Boosting Machine)

Non-parametric method, boosting through sequential trees but through **leaf wise method**
>> High speed

GOSS (Gradient-based One-Side Sampling): Focuses almost entirely on the data points the **model is struggling with** → faster without losing accuracy

EFB (Exclusive Feature Bundling): Bundles these sparse features together into a single feature to reduce the number of columns the model has to process.

Handles Categorical Data without encoding (Faster)

Higher ROC-AUC (0.8011) compared to XGBoost (0.7963, while maintaining similar False Positive (1458 VS 1504)

Balanced Threshold: 0.3585

```
--- LIGHTGBM FINAL TEST RESULTS ---
Optimal Equilibrium Threshold: 0.3585
Misclassification Error Rate:  0.2560
ROC AUC Score:                 0.8011

Confusion Matrix:
[[14275  3588]
 [ 3538  6430]]

Classification Report:
              precision    recall  f1-score   support

           0       0.80      0.80      0.80     17863
           1       0.64      0.65      0.64      9968

    accuracy                           0.74     27831
   macro avg       0.72      0.72      0.72     27831
weighted avg       0.74      0.74      0.74     27831
```

1000 trees, 31 leaves
***0.05 learning rate** (slower but more precise training), avoid overshooting the optimal solutions*
***Stopping rounds = 50** to save time and prevents overfitting*

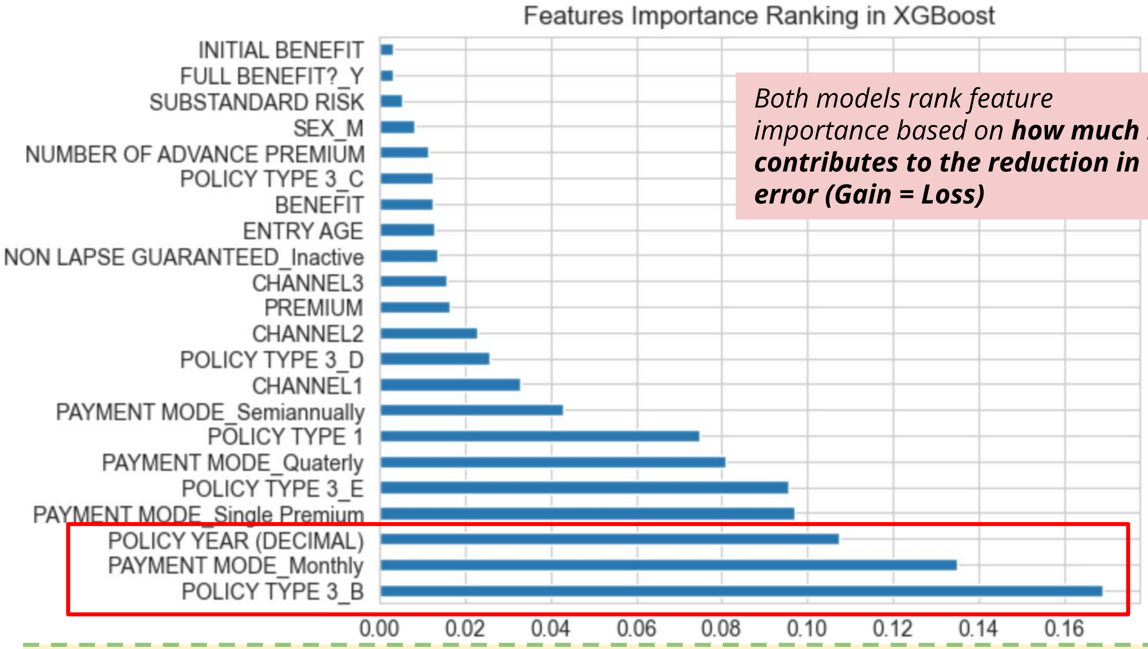
Bayes Classifier: 0.5

```
--- LIGHTGBM FINAL TEST RESULTS ---
Optimal Equilibrium Threshold: 0.5
Misclassification Error Rate:  0.2375
ROC AUC Score:                 0.8011

Confusion Matrix:
[[16405  1458]
 [ 5152  4816]]

Classification Report:
              precision    recall  f1-score   support

           0       0.76      0.92      0.83     17863
           1       0.77      0.48      0.59      9968

    accuracy                           0.76     27831
   macro avg       0.76      0.70      0.71     27831
weighted avg       0.76      0.76      0.75     27831
```

Trial and Error: 0.49 to 0.55 at 0.01 interval

Light GBM yields opposite features importance compared to XGBoost.

Features Importance in XGBoost



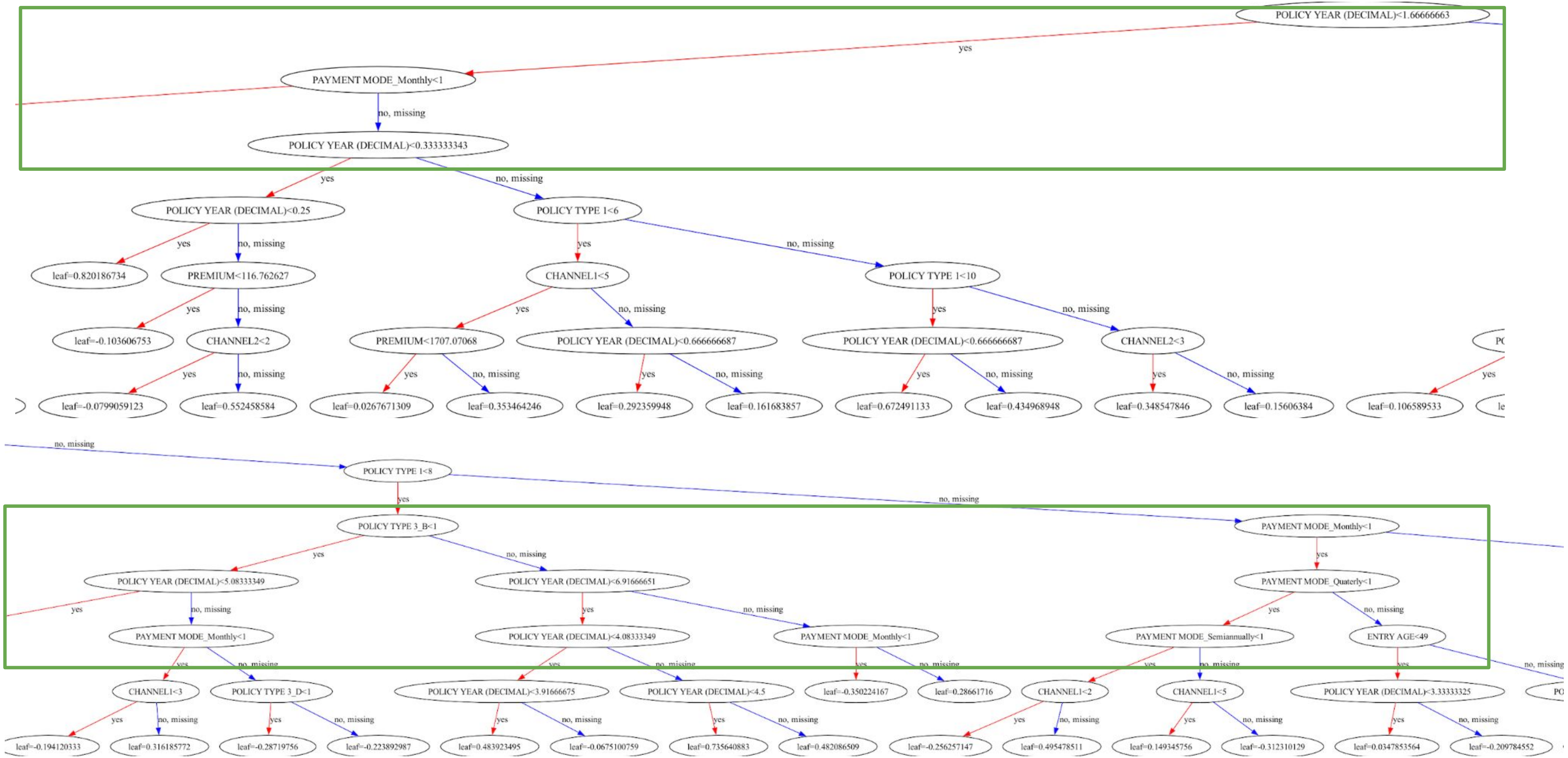
*Both models rank feature importance based on **how much it contributes to the reduction in error (Gain = Loss)***

Features Importance in Light GBM

feature	gain
PREMIUM	242268.114627
ENTRY AGE	36659.356031
NUMBER OF ADVANCE PREMIUM	36405.286278
POLICY TYPE 1	31319.837376
FULL BENEFIT?_Y	25801.726755
CHANNEL3	19637.840861
POLICY TYPE 3_E	18612.355959
INITIAL BENEFIT	12078.481295
PAYMENT MODE_Quaterly	8443.311542
CHANNEL1	6688.153684
SEX_M	5426.733157
POLICY YEAR (DECIMAL)	3530.212161
POLICY TYPE 3_B	2990.244941
PAYMENT MODE_Single Premium	2504.338411
CHANNEL2	2302.822327
PAYMENT MODE_Monthly	2143.918298
PAYMENT MODE_Semiannually	1155.993749
POLICY TYPE 3_D	425.400885
SUBSTANDARD RISK	305.809163
NON LAPSE GUARANTEED_Inactive	243.237051
BENEFIT	198.067721
POLICY TYPE 3_C	2.720383

XGBoost: Macro-Drivers (applies to every trees - which group lapse?)
Light GBM: Micro-Drivers (applies to a specific tree and its branch, which price point do they lapse? – Zoom in)
>> Individuals with Policy Type 3_B is most likely to lapse, and premium is the factor that cause **individuals in those group** to lapse *(actual zoom in factor)*

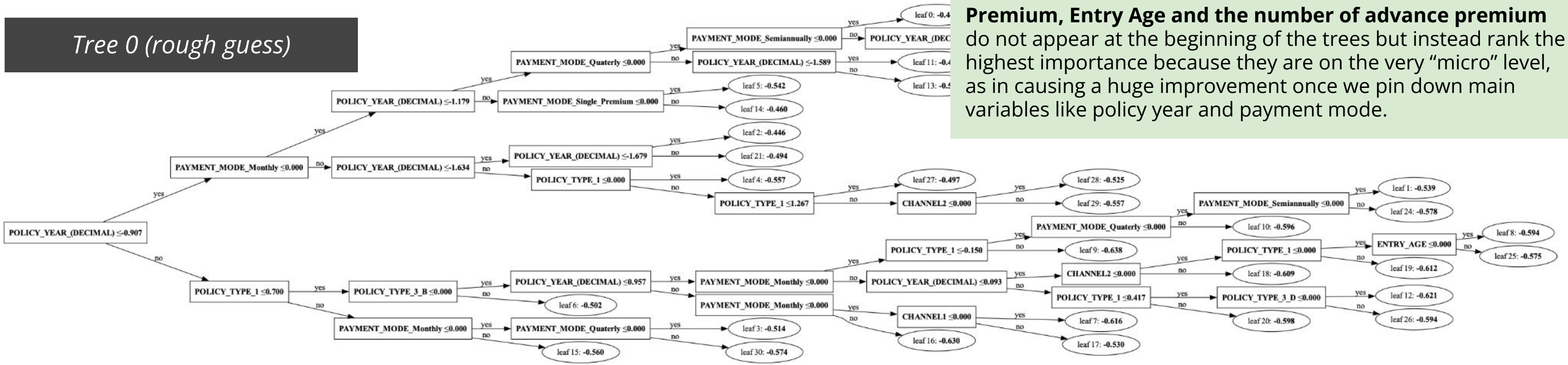
XGBoost:Policy Year and Payment Mode are the first two variables chosen to split the data, and are used repeatedly throughout the splitting



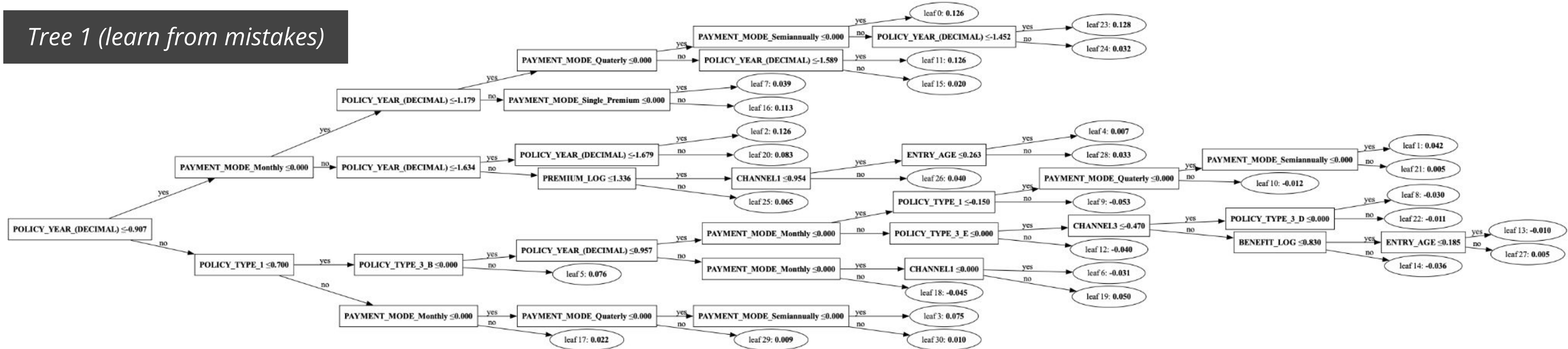
Light GBM: Started with Policy Year and Payment Mode for splitting, and slowly using other continuous variables (entry age,premium)

Premium, Entry Age and the number of advance premium do not appear at the beginning of the trees but instead rank the highest importance because they are on the very "micro" level, as in causing a huge improvement once we pin down main variables like policy year and payment mode.

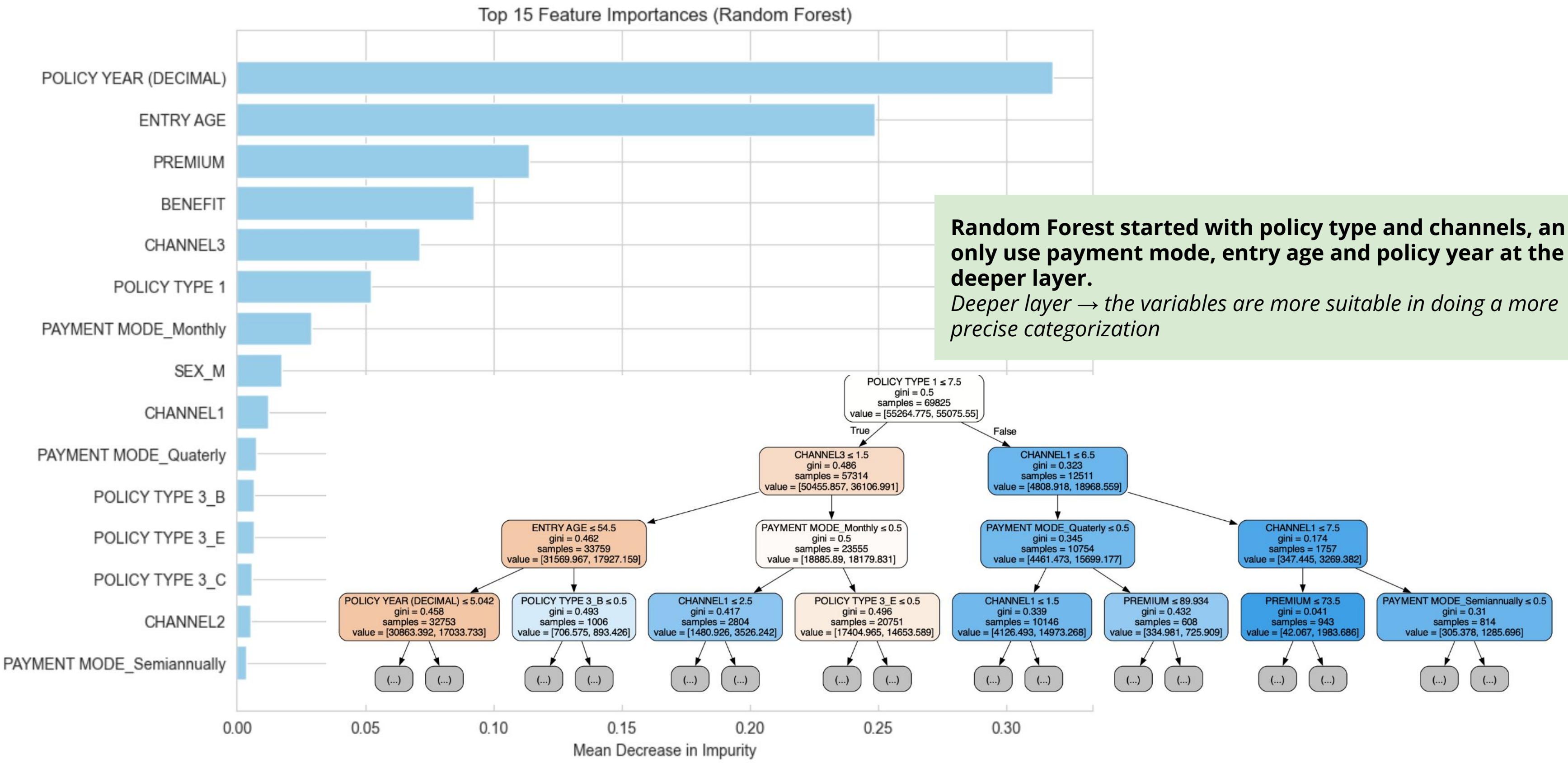
Tree 0 (rough guess)



Tree 1 (learn from mistakes)



Random Forest: A balanced of both XGBoost and Light GBM feature importance



Random Forest started with policy type and channels, an only use payment mode, entry age and policy year at the deeper layer.
Deeper layer → the variables are more suitable in doing a more precise categorization

Feature Importance Ranking

Model	Top 3 Important Features	Questions to be Answered
Random Forest	1. Policy Year 2. Entry Age 3. Premium	What are the **global risk factors** that determine the lapse risk? **RF rank the feature importances by averaging over many trees, hence this features categorization can be easily generalized.*
XGBoost	1. Policy Type_3B 2. Payment Mode 3. Policy Year	What are the **specific groups** of policyholders with higher/different profile of lapse risk?
Light GBM	1. Premium 2. Entry Age 3. Number of Advance Premium	**Within the specific group above, what are the specific point** that cause individuals to switch between CONTINUE and LAPSE? More micro-level/personal characteristics data

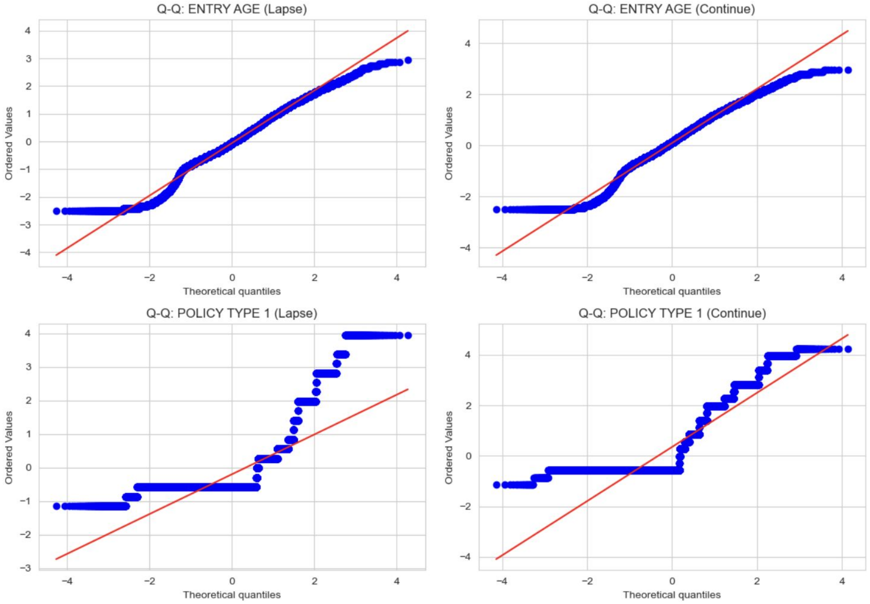
Regardless of specific niches, the fundamental lapse risk is a function of how long the customer has been with us and their demographic profile.

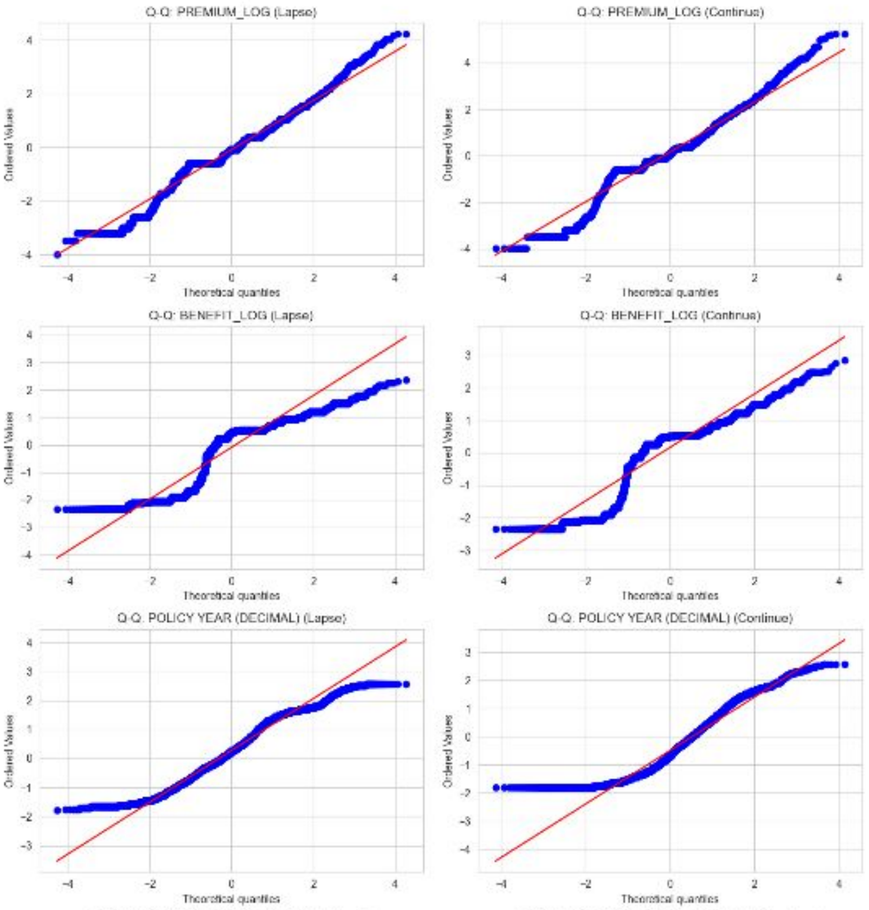
Move beyond generalities and pinpoint a specific 'High-Risk Profile': a monthly-paying customer on a ***Type 3_B policy***

For a specific age group on a ***Type 3_B policy,*** there is a specific ***price point or a lack of advance payments*** that acts as the final trigger for a lapse.

Linear/Quadratic Discriminant Analysis (LDA/QDA) - Assumption Check

The data violates the normality assumption required for LDA and QDA.
Features must be normally distributed within each class (LAPSE and CONTINUE)



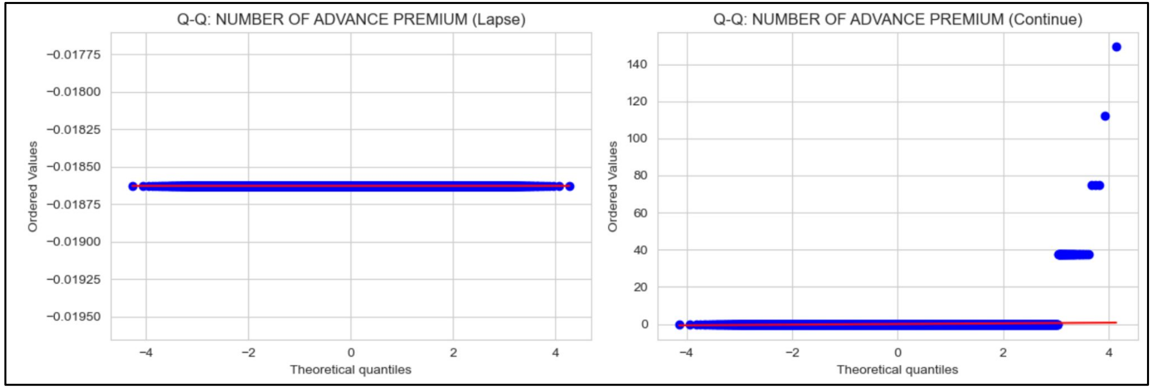


Linear/Quadratic Discriminant Analysis (LDA/QDA) - Assumption Check

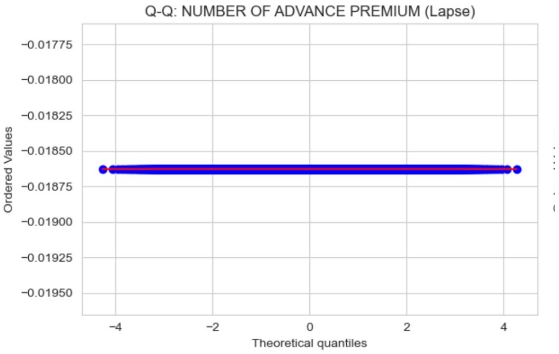
```
--- Shapiro-Wilk Test Results ---
                      Feature  Lapse_Stat  Lapse_P_Value  Continue_Stat  \
0                    CHANNEL1      0.6575            0.0         0.7213
1                    CHANNEL2      0.6697            0.0         0.6886
2                    CHANNEL3      0.6491            0.0         0.6568
3                   ENTRY AGE      0.9860            0.0         0.9845
4               POLICY TYPE 1      0.5840            0.0         0.7561
5            SUBSTANDARD RISK      0.0227            0.0         0.0342
6                     PREMIUM      0.2126            0.0         0.1113
7                     BENEFIT      0.4662            0.0         0.2323
8        POLICY YEAR (DECIMAL)     0.9821            0.0         0.9396
9    NUMBER OF ADVANCE PREMIUM     1.0000            1.0         0.0114
10             INITIAL BENEFIT     0.0027            0.0         0.0714

    Continue_P_Value  Normal_Lapse  Normal_Continue
0                0.0         False            False
1                0.0         False            False
2                0.0         False            False
3                0.0         False            False
4                0.0         False            False
5                0.0         False            False
6                0.0         False            False
7                0.0         False            False
8                0.0         False            False
9                0.0          True            False
10               0.0         False            False

Number of non-normal features: 11 out of 11
```

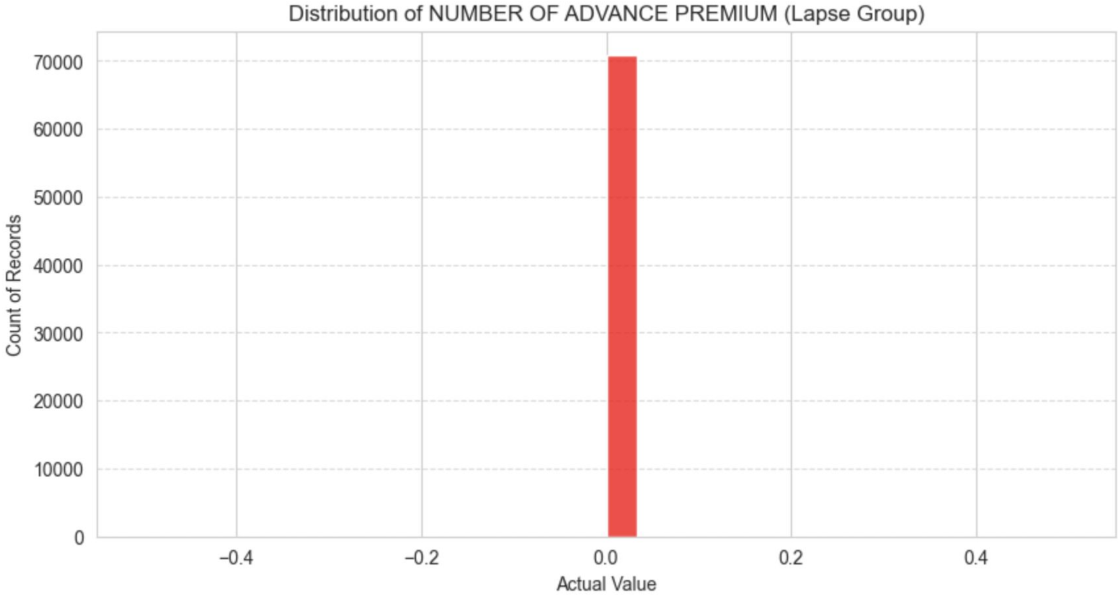
Through Shapiro Wilk test, we found Number of Advance Premium appear to be “normal” in the LAPSE category.

However through the QQ plot, it shows that the normality is fulfilled (line stays perfectly on the points) likely due to only one value exist for LAPSE.



Linear/Quadratic Discriminant Analysis (LDA/QDA) - Assumption Check





This justifies our guess! The reason why it appears to be normal (or 0) is because there is only 0 advance premium within the LAPSE group.

```
--- Frequency Counts for NUMBER OF ADVANCE PREMIUM (Lapse) ---
NUMBER OF ADVANCE PREMIUM
0.0    70854
Name: count, dtype: int64

Percentage of records that are exactly 0: 100.00%
```

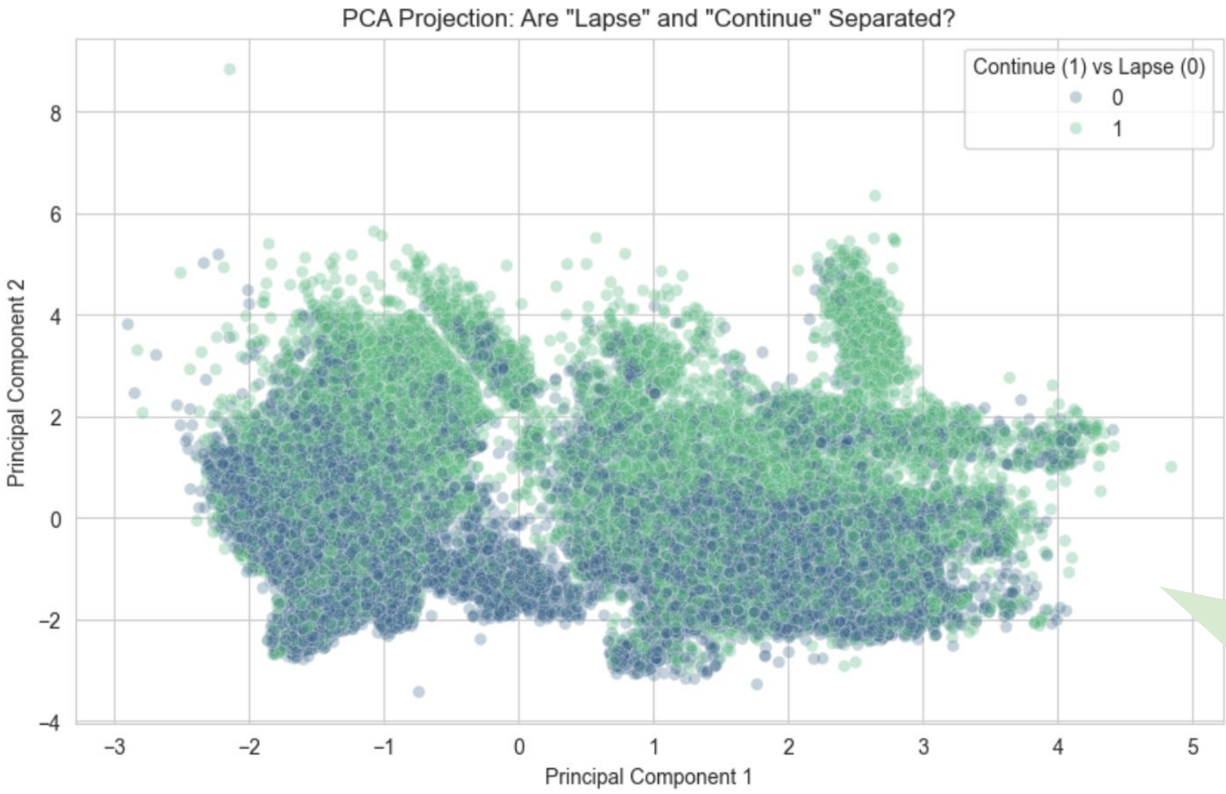
Linear/Quadratic Discriminant Analysis (LDA/QDA) - Assumption Check

```
Levene's Test for Equal Variance (p < 0.05 means Variances are DIFFERENT)
CHANNEL1             | p-value: 0.0000 | Violated (Use QDA)
CHANNEL2             | p-value: 0.0000 | Violated (Use QDA)
CHANNEL3             | p-value: 0.0000 | Violated (Use QDA)
ENTRY AGE            | p-value: 0.0000 | Violated (Use QDA)
POLICY TYPE 1        | p-value: 0.0000 | Violated (Use QDA)
SUBSTANDARD RISK     | p-value: 0.0000 | Violated (Use QDA)
PREMIUM_LOG          | p-value: 0.0000 | Violated (Use QDA)
BENEFIT_LOG          | p-value: 0.0000 | Violated (Use QDA)
POLICY YEAR (DECIMAL) | p-value: 0.0000 | Violated (Use QDA)
NUMBER OF ADVANCE PREMIUM | p-value: 0.0000 | Violated (Use QDA)
INITIAL BENEFIT_LOG  | p-value: 0.0000 | Violated (Use QDA)
```

We checked the second assumption: **Homoskedasticity is required to conduct LDA**, but all are violated hence suggesting QDA might be winning here.

Even though there is violation in assumptions in the real life it's hard for a data to fulfill all the assumptions for a specific test, hence **we will still conduct both LDA and QDA to see the performance and decide the better model from there.**

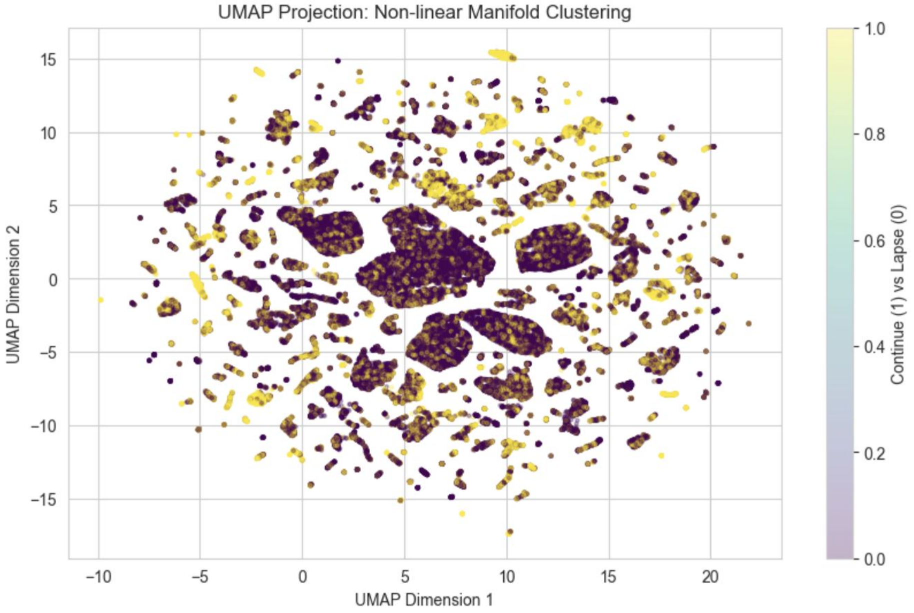
Preliminary Visualization Analysis (Principal Component)

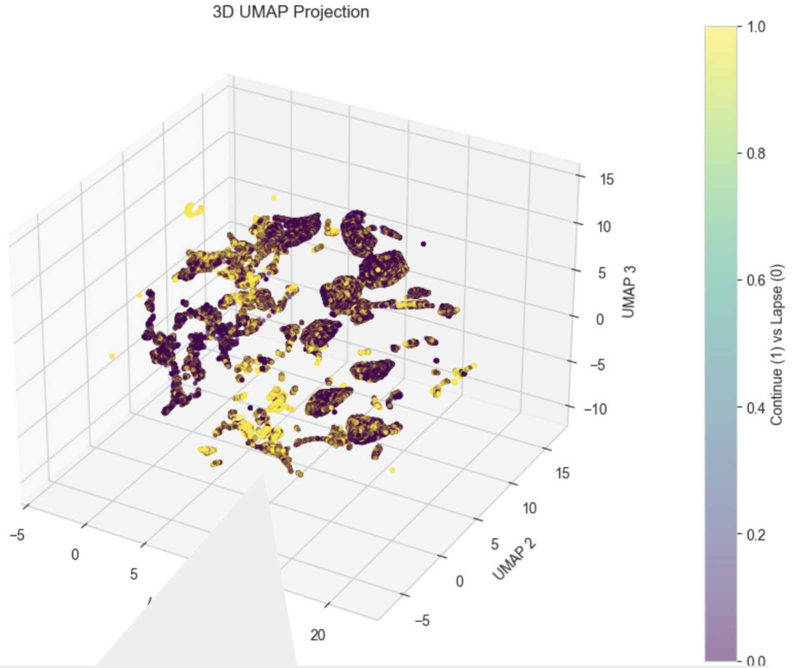



```
--- Cluster Purity Report ---
         Total Count  Continue Rate (%)
cluster
0              36276          26.995810
1              39697          35.899438
2              17405          19.672508
3                 47         100.000000
4              16969          70.864518
```

In a 2d Principal Component space, through linear combinations of predictors, all LAPSE and CONTINUE categories are highly overlapping between each other → **a sign that might be hard to figure out a boundary that best separate the two classes.**

Preliminary Visualization Analysis (UMAP - Non-linear mapping)

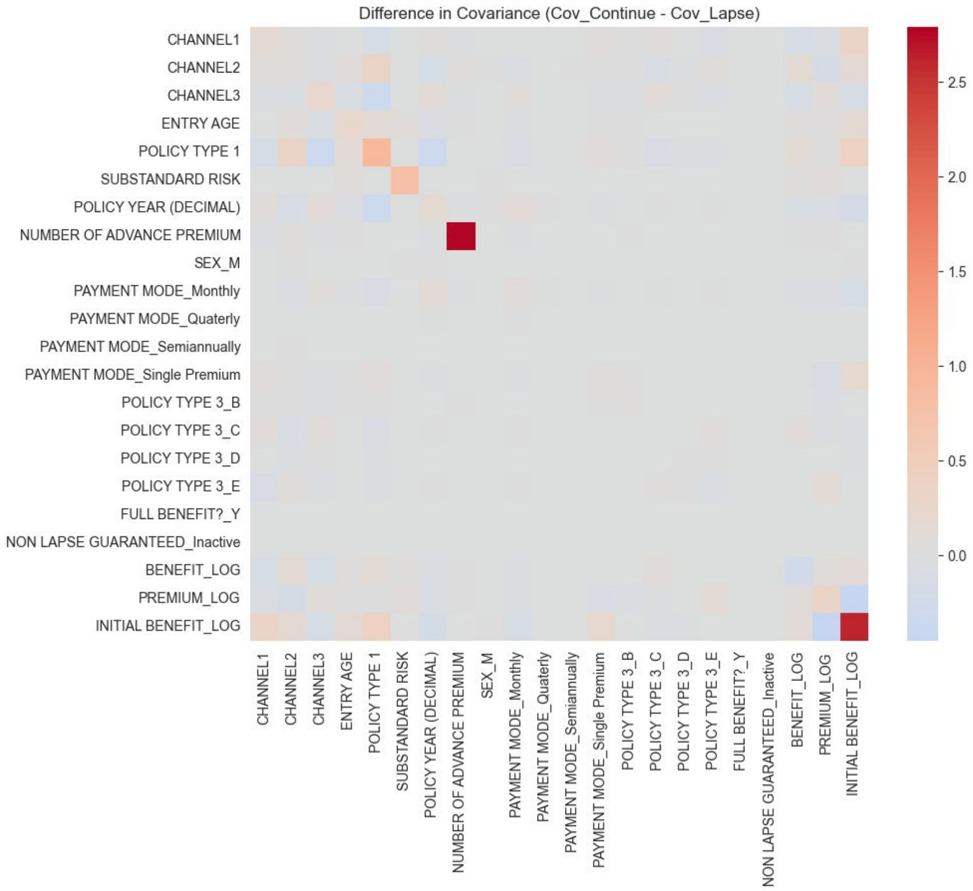




Imagine all data sitting on a high dimensional space (manifold), **connects points that are close to each other and build a web of the data's structure** → then recreate the same web/structure to explore hidden segments

Purple 'Lapse' clusters tightly packed together confirms that **lapse behavior is concentrated in specific risk segments**—validating our XGBoost profiling. However, the presence of yellow dots within those same clusters shows that there is a **fine-grained 'Micro-Trigger'**—like a specific Premium threshold—that ultimately decides the outcome, which explains our LightGBM results.

Linear/Quadratic Discriminant Analysis (LDA/QDA) - Assumption Check



For QDA to work perfectly, it requires that each class has its own unique covariance matrix → Cov(Continue) - Cov(Lapse) should be **having large differences**

Most of the **differences** are very small and close to 0, hence the model is only picking up the statistical noise.

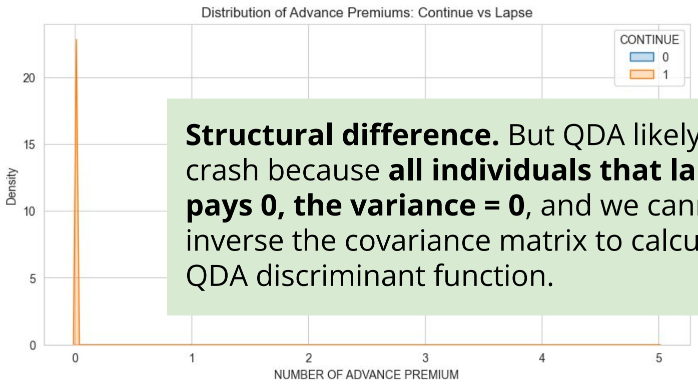
When the covariance differences are very small, this suggests that **LDA might work better here** since LDA assumes that both classes share the same covariance matrix

White region: the math is broken/difference in covariance are undefined

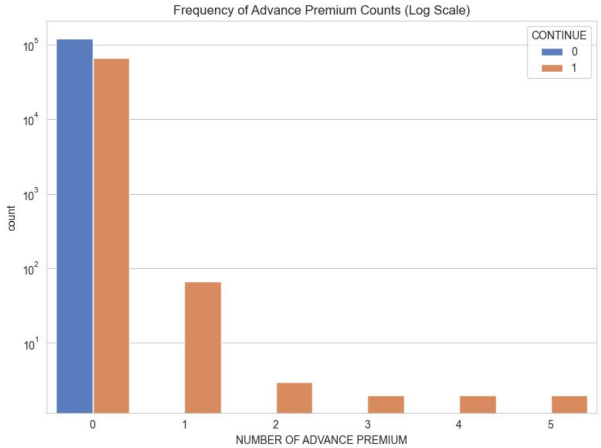
A very strong difference in covariance for number of advance premium and initial benefit

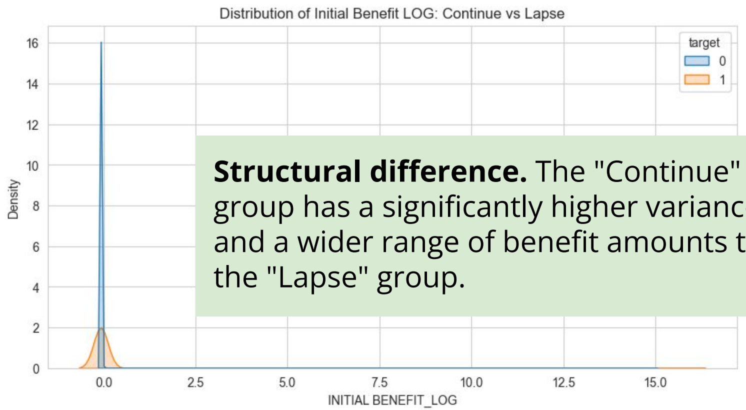
High Covariance Differences are due to structural differences.

Distribution of Advance Premium	Distribution of Initial Benefit_LOG

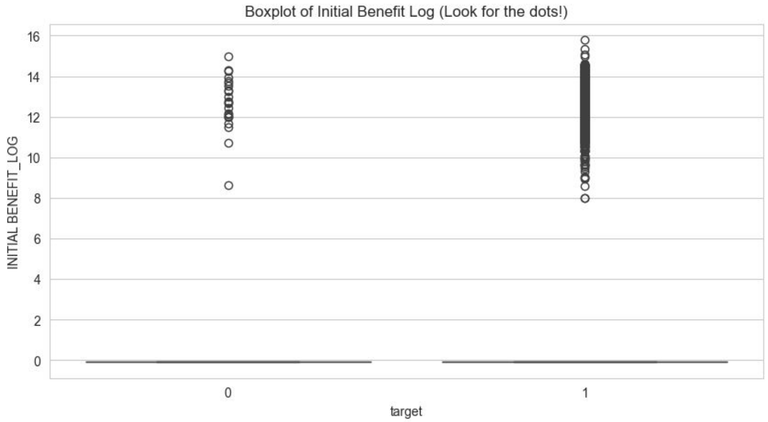


Structural difference. But QDA likely to crash because **all individuals that lapse pays 0, the variance = 0**, and we cannot inverse the covariance matrix to calculate QDA discriminant function.





Structural difference. The "Continue" group has a significantly higher variance and a wider range of benefit amounts than the "Lapse" group.



High covariance difference is due to 0 variance - QDA, Naive Bayes crash

--- Variance per Predictor by Class ---

	Variance_Lapse (0)	Variance_Continue (1)
NUMBER OF ADVANCE PREMIUM	0.000000	2.790292
FULL BENEFIT?_Y	0.000155	0.001364
PAYMENT MODE_Single Premium	0.000733	0.036615
NON LAPSE GUARANTEED_Inactive	0.001902	0.000733
POLICY TYPE 3_B	0.005055	0.040916
PAYMENT MODE_Semiannually	0.025562	0.030970
INITIAL BENEFIT_LOG	0.053300	2.667809
PAYMENT MODE_Quaterly	0.055664	0.046270
POLICY TYPE 3_D	0.083692	0.097855
POLICY TYPE 3_E	0.101437	0.037409
PAYMENT MODE_Monthly	0.138703	0.232390
SEX_M	0.243055	0.245201
POLICY TYPE 3_C	0.243161	0.227808
POLICY TYPE 1	0.601388	1.523841
SUBSTANDARD RISK	0.723641	1.494545
POLICY YEAR (DECIMAL)	0.809874	0.961555
PREMIUM_LOG	0.866554	1.196932
ENTRY AGE	0.912593	1.141517
CHANNEL3	0.916719	1.145586
CHANNEL1	0.938038	1.083483

--- Features with potentially problematic low variance (< 0.01) ---

	Variance_Lapse (0)	Variance_Continue (1)
NUMBER OF ADVANCE PREMIUM	0.000000	2.790292
PAYMENT MODE_Single Premium	0.000733	0.036615
POLICY TYPE 3_B	0.005055	0.040916
FULL BENEFIT?_Y	0.000155	0.001364
NON LAPSE GUARANTEED_Inactive	0.001902	0.000733

QDA: Matrix Singularity Problem
These models are "parametric." They assume your data follows a Gaussian (Normal) distribution. To calculate the probability of a lapse, **the model must invert the Covariance Matrix.**

If a feature has **zero variance,** that matrix becomes singular (like trying to divide by zero in basic math). The computer literally cannot complete the calculation.
**LDA won't suffer much because LDA assumes the same covariance matrix for all classes and will calculate pooled variance (non-zero).*

Naive Bayes: Certainty Trap
Naive Bayes calculates the final probability by multiplying the **probabilities of each individual feature together.**

If a feature has zero variance in the training set (e.g., if every observed "Lapser" paid monthly), the model assigns a 0% probability to any other value. Because multiplying, that one "0" wipes out all other evidence, no matter how strong.

LDA might works better than QDA here.

--- Absolute Mean Difference per Feature ---	
POLICY YEAR (DECIMAL)	0.768631
POLICY TYPE 1	0.544814
CHANNEL1	0.207374
PAYMENT MODE_Monthly	0.200889
ENTRY AGE	0.153710
CHANNEL3	0.075956
POLICY TYPE 3_E	0.075637
POLICY TYPE 3_C	0.066269
CHANNEL2	0.063668
NUMBER OF ADVANCE PREMIUM	0.052007

Diff-in-Covariance plot shows the shape/spread of the data points around CONTINUE vs LAPSE variables. *(Shape)*

The absolute mean difference per feature above shows the difference in the mean of the specific variable between two classes. *(Location) → the mean difference is measured in the* ***unit of the variables themselves***

LDA and QDA are both "centroid-based" classifiers. They calculate the center (mean) of each class and try to determine which center a new data point is closer to.

- **In LDA:** The Mean Difference is the **only** thing that creates the "slope" of the decision boundary. If the mean difference is 0, LDA is essentially flipping a coin; it cannot separate the classes because it assumes they both look like the same cloud of points, just centered in different spots.
- **In QDA:** Mean differences are still important, but they can be "overridden" by the shape of the data **(*Difference in Covariance is more important)***

No significant differences in covariance between classes but there is some differences in mean available → **LDA might be a better option here**

*Concern: Homoskedasticity is violated, no clear linear boundary between two classes *(Not a big deal since it is very difficult for real-life messy data to fulfill all assumptions perfectly).*

Linear/Quadratic Discriminant Analysis (LDA/QDA) - Model Fitting

```
--- LDA FINAL TEST RESULTS ---
Optimal Equilibrium Threshold: 0.3618
Misclassification Error Rate:  0.2817
ROC AUC Score:                 0.7590

Confusion Matrix:
[[13864  3999]
 [ 3840  6128]]

Classification Report:
              precision    recall  f1-score   support

           0       0.78      0.78      0.78     17863
           1       0.61      0.61      0.61      9968

    accuracy                           0.72     27831
   macro avg       0.69      0.70      0.69     27831
weighted avg       0.72      0.72      0.72     27831
```

```
--- QDA FINAL TEST RESULTS ---
Optimal Equilibrium Threshold: 0.0014
Misclassification Error Rate:  0.2870
ROC AUC Score:                 0.7467

Confusion Matrix:
[[13852  4011]
 [ 3977  5991]]

Classification Report:
              precision    recall  f1-score   support

           0       0.78      0.78      0.78     17863
           1       0.60      0.60      0.60      9968

    accuracy                           0.71     27831
   macro avg       0.69      0.69      0.69     27831
weighted avg       0.71      0.71      0.71     27831
```

Threshold for QDA is more **unrealistic and dangerous** compared to LDA.

Linear/Quadratic Discriminant Analysis (LDA/QDA) - Model Fitting

<u>Lower</u> ROC-AUC (0.7590) compared to Light GBM (0.8),
But having lower False Positive (1969 VS 1458)

Balanced Threshold: 0.3380

```
--- LDA FINAL TEST RESULTS ---
Optimal Equilibrium Threshold: 0.3618
Misclassification Error Rate:  0.2817
ROC AUC Score:                 0.7590

Confusion Matrix:
[[13864  3999]
 [ 3840  6128]]

Classification Report:
              precision    recall  f1-score   support

           0       0.78      0.78      0.78     17863
           1       0.61      0.61      0.61      9968

    accuracy                           0.72     27831
   macro avg       0.69      0.70      0.69     27831
weighted avg       0.72      0.72      0.72     27831
```

Bayes Classifier: 0.5

```
--- LDA FINAL TEST RESULTS ---
Optimal Equilibrium Threshold: 0.5
Misclassification Error Rate:  0.2609
ROC AUC Score:                 0.7590

Confusion Matrix:
[[15894  1969]
 [ 5291  4677]]

Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.89      0.81     17863
           1       0.70      0.47      0.56      9968

    accuracy                           0.74     27831
   macro avg       0.73      0.68      0.69     27831
weighted avg       0.73      0.74      0.72     27831
```

LDA/QDA/Naive Bayes - Leaving out Advanced Premium

```
========== LDA FINAL TEST RESULTS ==========
Optimal Threshold: 0.3610
ROC AUC Score:     0.7591
Error Rate:        0.2815

Confusion Matrix:
[[13869  3994]
 [ 3840  6128]]
```

```
========== QDA FINAL TEST RESULTS ==========
Optimal Threshold: 0.0231
ROC AUC Score:     0.7467
Error Rate:        0.2870

Confusion Matrix:
[[13851  4012]
 [ 3976  5992]]
```

```
========== Naive Bayes FINAL TEST RESULTS ==========
Optimal Threshold: 0.0019
ROC AUC Score:     0.7392
Error Rate:        0.2904

Confusion Matrix:
[[13844  4019]
 [ 4064  5904]]
```

Leaving out Advanced Premium, not much changes in ROC AUC score, error rate and False Positive.

QDA and Naive Bayes still very unstable/worse performance (very low optimal threshold)

Not really useful in leaving out advanced premium, hence we will use back the original models with all variables in.

Feature Importance by Classes through LDA (Full Model)

--- Feature Means by Class with Absolute Differences ---

	Lapse (0)	Continue (1)	Abs_Difference
POLICY YEAR (DECIMAL)	0.275302	-0.493329	0.768631
POLICY TYPE 1	-0.195137	0.349677	0.544814
PREMIUM_LOG	-0.091896	0.164674	0.256570
BENEFIT_LOG	-0.090684	0.162502	0.253187
INITIAL BENEFIT_LOG	-0.075747	0.135735	0.211481
CHANNEL1	-0.074276	0.133099	0.207374
PAYMENT MODE_Monthly	0.833616	0.632726	0.200889
ENTRY AGE	-0.055054	0.098655	0.153710
CHANNEL3	-0.027205	0.048751	0.075956
POLICY TYPE 3_E	0.114560	0.038923	0.075637
POLICY TYPE 3_C	0.417281	0.351012	0.066269
CHANNEL2	0.022804	-0.040864	0.063668
NUMBER OF ADVANCE PREMIUM	-0.018627	0.033379	0.052007
POLICY TYPE 3_B	0.005081	0.042742	0.037661
PAYMENT MODE_Single Premium	0.000734	0.038063	0.037329
SUBSTANDARD RISK	-0.012112	0.021704	0.033816
POLICY TYPE 3_D	0.092190	0.109939	0.017750
SEX_M	0.583355	0.569322	0.014032
PAYMENT MODE_Quaterly	0.059164	0.048634	0.010530
PAYMENT MODE_Semiannually	0.026251	0.031993	0.005742
FULL BENEFIT?_Y	0.000155	0.001366	0.001210
NON LAPSE GUARANTEED_Inactive	0.998095	0.999267	0.001172

In LDA, because we assume a shared covariance, **the variables at the top of this list are the ones doing the heavy lifting** to pull the two classes apart.

**The means are standardized*

Findings

Policy Year (Decimal)

- Most people stay for the first few months/years (keeping the "Continue" mean low).
- Lapses often cluster around the 1-year, 2-year, or 5-year mark when premiums increase or the "newness" wears off.
- This pushes the **average age of a "Lapse" policy higher than the average age of an "Active" policy.**

>> People who stay tend to have younger policies
>> People who leave tend to have older policies

Neural Network

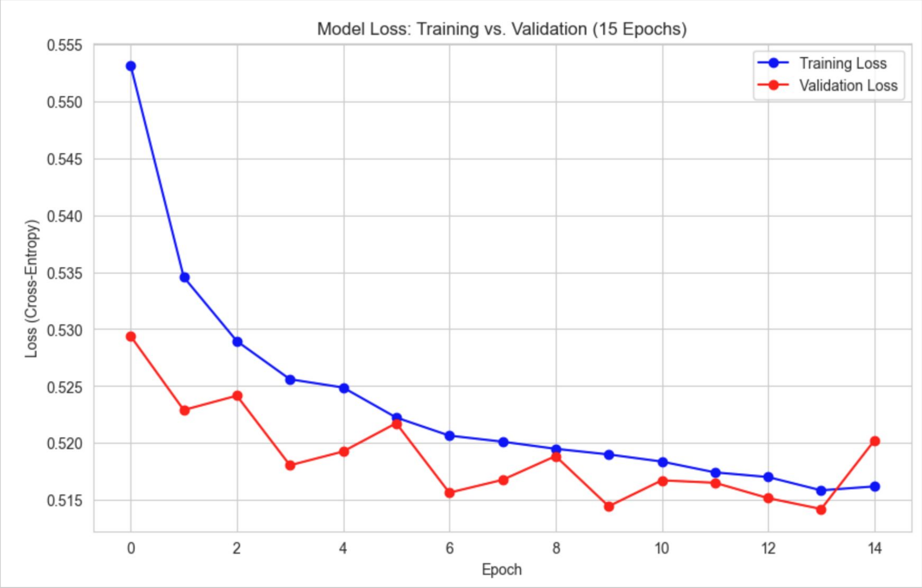
Steps

Layers: 128 → 64 → 32 → 2 layers
Activation: reLu for 128, 64 and 32; Softmax for 2 layers
Dropout = 0.5 before layer 2
Randomly shut down 50% of the neurons to force the model relearn the pattern and prevent overfitting

Compile:
Optimizer: adam → *updates weights to minimize the error. If the weights are far from the goal, it takes big steps; as it gets closer, it takes smaller, precise steps.*
Loss: *sparse categorical cross entropy → calculate how wrong the model is (sparse = binary)*

Early stop:
If the validation loss does not improve for 10 epochs in a row, kill the training to avoid overfitting
Restore best weights: restore to the best number of epochs that minimizes training and validation error

Epochs = 15; Batch size = 32
Allow the model to learn the data 15 times, smaller batch size to add noise for better generalization



Both training and validation error loss converges at 13 epoch, and start overfitting (validation loss spike) at epoch 14.
Note due to the randomization nature of neural network fitting, the best epoch that minimizes the losses is normally between 12 and 13 epochs.

Neural Network

Designed to automatically learn **complex, non-linear relationships** in **multidimensional** data through **multiple layers** of abstraction.

Input Layer: Receives features

Hidden Layers (Learning): Each layer performs a mathematical transformation (weighted linear combination)m followed by an Activation Function (like ReLU) → capture "interaction effects" that are too complex for humans to manually program.

Output Layer: Softmax layer with 2 units, which outputs the probability of a customer belonging to either "Class 0" (Lapse) or "Class 1" (Continue).

Lower ROC-AUC (0.7829) compared to Light GBM (0.8),
But having lower False Positive (1193 VS 1458)

Balanced Threshold: 0.3380

```
--- NEURAL NETWORK FINAL TEST RESULTS ---
Optimal Equilibrium Threshold: 0.3380
Misclassification Error Rate:  0.2663
ROC AUC Score:                 0.7829

Confusion Matrix:
[[14133  3730]
 [ 3682  6286]]

Classification Report:
              precision    recall  f1-score   support

           0       0.79      0.79      0.79     17863
           1       0.63      0.63      0.63      9968

    accuracy                           0.73     27831
   macro avg       0.71      0.71      0.71     27831
weighted avg       0.73      0.73      0.73     27831
```

Bayes Classifier: 0.5

```
--- NEURAL NETWORK FINAL TEST RESULTS ---
Optimal Equilibrium Threshold: 0.5
Misclassification Error Rate:  0.2494
ROC AUC Score:                 0.7829

Confusion Matrix:
[[16670  1193]
 [ 5748  4220]]

Classification Report:
              precision    recall  f1-score   support

           0       0.74      0.93      0.83     17863
           1       0.78      0.42      0.55      9968

    accuracy                           0.75     27831
   macro avg       0.76      0.68      0.69     27831
weighted avg       0.76      0.75      0.73     27831
```

Neural Network has similar feature importance as Light GBM.

```
--- NEURAL NETWORK FEATURE IMPORTANCE ---
PREMIUM                              0.117689
NUMBER OF ADVANCE PREMIUM            0.035749
FULL BENEFIT?_Y                      0.023605
POLICY TYPE 1                        0.018374
CHANNEL1                             0.014911
POLICY TYPE 3_B                      0.011889
ENTRY AGE                            0.010704
INITIAL BENEFIT                      0.008305
CHANNEL2                             0.007997
PAYMENT MODE_Single Premium          0.006893
PAYMENT MODE_Quaterly                0.006678
POLICY TYPE 3_E                      0.005622
PAYMENT MODE_Semiannually            0.004182
CHANNEL3                             0.003697
SEX_M                                0.002370
SUBSTANDARD RISK                     0.000914
PAYMENT MODE_Monthly                 0.000580
NON LAPSE GUARANTEED_Inactive        0.000379
BENEFIT                              0.000241
POLICY YEAR (DECIMAL)                0.000169
POLICY TYPE 3_D                      0.000089
POLICY TYPE 3_C                      0.000012
```

Rank by Permutation Importance

1. Keeps the variable in the model but turns it into "noise"—it preserves the range of the data but breaks its relationship with the outcome.
2. The model then re-runs the data through the network.
3. The values = drop in accuracy

Model	Top 3 Important Features
Random Forest	1. Policy Year 2. Entry Age 3. Premium
XGBoost	1. Policy Type_3B 2. Payment Mode 3. Policy Year
Light GBM	1. Premium 2. Entry Age 3. Number of Advance Premium
Neural Network	1. Premium 2. Number of Advance Premium 3. Full Benefit (*Binary*)

General Criteria

Group-specific Macro-factors

Micro-factors

NN is highly sensitive and good at capturing interaction effect/ non-linear patterns - hence they captures micro-drivers similar to Light GBM

ROC-AUC and False Positives across models

Model *Rank decreasing order of ROC AUC*	**ROC AUC** *Model Precision*	**False Positives** *Minimize because expensive (0.5 threshold)*
Light GBM	0.8011	1458
XGBoost	0.7963	1539
Neural Network	0.7829	1193
KNN (K=81)	0.7756	1504
Logistics Regression w/Interaction	0.77	1966
Random Forest Classifier	0.7693	2834
Logistics Regression (Base)	0.7599	2008
LDA	0.7590	1969

Ensemble Model (0.6*Light GBM + 0.4*Neural Network)

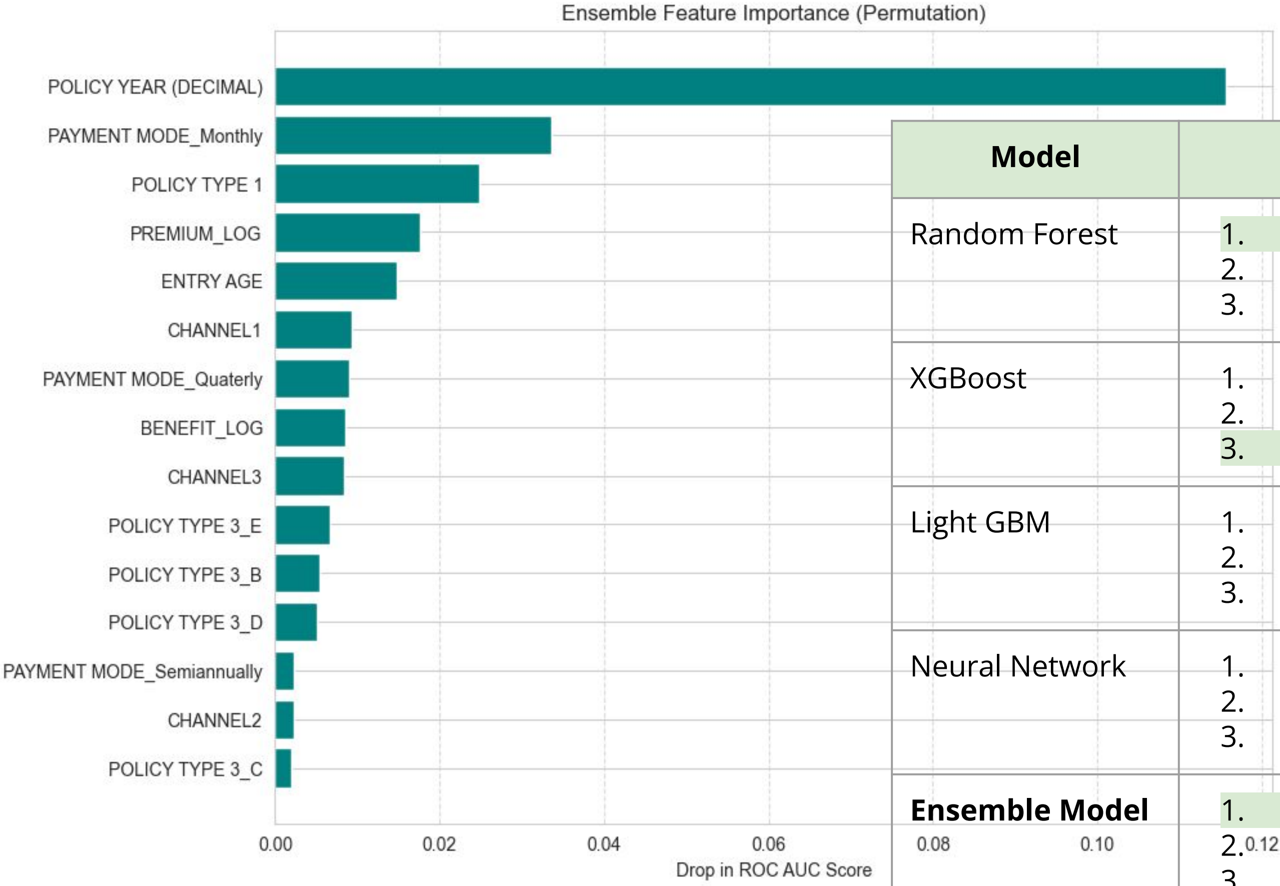
Model *Rank decreasing order of ROC AUC*	**ROC AUC** *Model Precision*	**False Positives** *Minimize because expensive (0.5 threshold)*
Light GBM	0.8011	1458
0.6*Light GBM + 0.4*Neural Network	0.7986	1426
XGBoost	0.7963	1539
Neural Network	0.7829	1193

```
--- ENSEMBLE MODEL PERFORMANCE ---
ROC AUC Score           : 0.7986
Misclassification Error : 0.2381
---------------------------------
True Negatives (TN)     : 16437
False Positives (FP)    : 1426   <-- Target to minimize
False Negatives (FN)    : 5200
True Positives (TP)     : 4768
---------------------------------
Total Observations      : 27831
```

Reduce the False Positives from Light GBM without losing much of the ROC AUC

We want to have the best of both high ROC AUC and low False Positives, hence initially test with 50/50. But 60/40 could improve ROC AUC without increasing False Positives.

Feature Importance (0.6*Light GBM + 0.4*Neural Network)



Model	Top 3 Important Features
Random Forest	1. Policy Year 2. Entry Age 3. Premium
XGBoost	1. Policy Type_3B 2. Payment Mode (Monthly) 3. Policy Year
Light GBM	1. Premium 2. Entry Age 3. Number of Advance Premium
Neural Network	1. Premium 2. Number of Advance Premium 3. Full Benefit (*Binary)*
Ensemble Model	1. Policy Year 2. Payment Mode (Monthly) 3. Policy Type 1

Conclusion and Limitation

- *What are the conclusion we can derive from this experiment? Which is the best prediction model?*

- *What are the limitations and the next steps?*

Conclusion

Goal

Determine the most effective mathematical approach for predicting life insurance lapse while identifying the core behavioral and demographic triggers.

Ensemble Model superiority for business logic

While **LightGBM** achieved the highest standalone **ROC AUC (0.8011)** , the **Ensemble model** is the recommended final model.

This approach reduced False Positives to **1,426** (down from LightGBM's 1,458) while maintaining a high **ROC AUC of 0.7986**.

This balance is critical because False Positives are "expensive," leading to wasted retention resources on loyal customers.

A hierarchy of global, macro and micro lapse drivers

Macro-Drivers (The "Who"): Policy Year (Decimal) and Payment Mode (Monthly) are the most fundamental indicators of risk. Shorter policy tenure and higher payment frequency provide more "opportunities" for a customer to stop paying.

Micro-Drivers (The "Tipping Point"): Within high-risk groups, financial variables like **Premium, Entry Age, and the Number of Advance Premiums** act as the final triggers. The absence of advance premiums is a nearly absolute predictor of lapse in certain segments.

Algorithmic Insights

Traditional statistical models like LDA and Naive Bayes were limited by the data's **violation of normality and the presence of zero-variance predictors** (e.g., Advance Premiums in the lapse group).

Conversely, **non-parametric tree-based models and Neural Networks** successfully captured the non-linear, structural "islands" of risk visualized in UMAP projections.

Limitation

Limitation	Impact on Analysis	Mitigation
Domain Blindness: Metadata Gap	The dataset lacked clear definitions for several key features, specifically **Channel 1, 2, and 3 and Policy Type 1, 2, and 3** → Lack interpretability.	For future analysis, look for clearly defined data dictionary to understand the definition and categories.
Assumption Violations	**Most continuous predictors violated the normality assumption** required for LDA and QDA and failed the **Box-Tidwell test** for linearity in Logistic Regression.	Implement **Generalized Additive Models (GAMs)** → allow the model to learn non-linear "smooth" shapes for each predictor automatically + ensure interpretability
Numerical Instability	The "Number of Advance Premium" feature had exactly 0% variance within the Lapse group → mathematical failure in **QDA (Matrix Singularity) and a "Certainty Trap" in Naive Bayes.**	Regularization, add noise to the covariance matrix diagonal to prevent becoming singular (non-zero).
Classification Threshold	All classification threshold for False Positive is only based on 0.5 (Naive threshold) and maximizing ROC AUC	Construct a **Cost-Benefit Utility Function** and maximize that function instead of only focusing on ROC AUC; a more optimum threshold(?)
Temporal/External Dynamics	The analysis is based on a static data snapshot, ignores how lapse risk might change due to **external economic shifts.**	Integrate **economic indices** such as inflation and income changes at a more personalized level